

CAPÍTULO 1: Fundamentos de programación

B2.1.1 - Introducción

La programación consiste en diseñar una serie de instrucciones que una computadora puede ejecutar para resolver un problema.

Actualmente existen cientos de lenguajes de programación, como Python, Java, C++, JavaScript y C#. Todos ellos permiten desarrollar programas, aunque cada uno posee características particulares y aplicaciones específicas.

En este curso utilizaremos **Python**, uno de los lenguajes más populares del mundo debido a su sintaxis sencilla, su gran cantidad de aplicaciones y su utilización tanto en el ámbito educativo como profesional.

Antes de aprender estructuras más complejas, es necesario comprender los conceptos fundamentales de la programación. En este capítulo aprenderemos cómo almacenar información, manipular distintos tipos de datos y comprender el funcionamiento de las variables, que constituyen la base de cualquier programa.

1. ¿Qué es un algoritmo?

Antes de escribir un programa debemos diseñar un **algoritmo**.

Definición

Un algoritmo es un conjunto finito y ordenado de instrucciones que permiten resolver un problema o realizar una tarea.

Un algoritmo debe cumplir cuatro características fundamentales:

- Tener un inicio.
 - Tener un final.
 - Estar compuesto por pasos ordenados.
 - Resolver un problema.
-

Ejemplo cotidiano

Supongamos que queremos preparar un mate.

El algoritmo podría ser:

1. Buscar el mate.
2. Colocar la yerba.
3. Calentar el agua.
4. Servir el agua.
5. Tomar el mate.

Si cambiamos el orden de los pasos:

1. Tomar el mate.
2. Buscar el mate.

el algoritmo deja de tener sentido.

En programación ocurre exactamente lo mismo.

2. ¿Qué es un programa?

Un programa es la implementación de un algoritmo utilizando un lenguaje de programación.

Por ejemplo:

Algoritmo

```
Pedir dos números.  
Sumarlos.  
Mostrar el resultado.
```

Programa en Python

```
numero1 = int(input("Ingrese un número: "))  
numero2 = int(input("Ingrese otro número: "))  
  
suma = numero1 + numero2  
  
print("Resultado:", suma)
```

El algoritmo describe **qué debe hacerse**, mientras que el programa indica **cómo hacerlo utilizando un lenguaje de programación**.

3. ¿Qué es una variable?

Definición

Una variable es un espacio de memoria reservado para almacenar información que puede cambiar durante la ejecución de un programa.

El nombre "variable" proviene justamente de que su contenido puede variar.

Por ejemplo:

```
edad = 16
```

Más adelante:

```
edad = 17
```

La variable sigue siendo la misma, pero el dato almacenado cambió.

Una analogía

Imaginemos un conjunto de cajas etiquetadas.

```
+-----+
|  edad  |
+-----+
|   16   |
+-----+
```

La etiqueta de la caja representa el nombre de la variable.

El contenido representa el valor almacenado.

En cualquier momento podemos reemplazar el contenido.

```
+-----+
|  edad  |
+-----+
|   17   |
+-----+
```

¿Cómo crea Python una variable?

En muchos lenguajes es necesario declarar el tipo de dato antes de utilizar una variable.

Por ejemplo, en Java:

```
int edad = 16;
```

Python funciona de otra manera.

La variable se crea automáticamente cuando se le asigna un valor.

```
edad = 16
```

Python detecta que 16 es un entero y asigna automáticamente el tipo correspondiente.

Esta característica hace que Python sea un lenguaje de **tipado dinámico**.

Asignación

El signo "=" **NO significa igualdad matemática**.

En programación significa:

Guardar el valor de la derecha dentro de la variable de la izquierda.

Ejemplo

```
x = 5
```

No se está diciendo que "x es igual a cinco".

Se está indicando:

"Guardar el número cinco dentro de la variable llamada x."

Otro ejemplo

```
x = 5
```

```
x = 8
```

Después de ejecutarse la segunda línea, el valor 5 desaparece.

La variable ahora contiene únicamente el número 8.

Visualización de una variable

```
x = 25
```

Variable

x

```
+-----+  
| 25    |  
+-----+
```

Después

```
x = 40
```

Variable

x

```
+-----+  
| 40    |  
+-----+
```

El valor anterior fue reemplazado.

Asignaciones sucesivas

Observemos el siguiente programa.

```
x = 5  
y = x  
x = 20  
  
print(x)  
print(y)
```

Antes de ejecutar:

x

vacía

```
y  
vacía
```

Después de la primera línea

```
x  
5
```

Después de la segunda

```
x -> 5  
y -> 5
```

Después de la tercera

```
x -> 20  
y -> 5
```

Salida

```
20  
5
```

Es importante comprender que **las variables almacenan valores independientes.**

Reglas para nombrar variables

Python posee reglas específicas.

Correcto

```
edad = 17  
nombre = "Ana"  
precio_total = 2500  
cantidadAlumnos = 32  
_promedio = 8
```

Incorrecto

```
2edad = 15
```

No puede comenzar con un número.

```
precio total = 20
```

No puede contener espacios.

```
mi-variable = 30
```

No puede contener guiones.

```
class = 5
```

No puede utilizar palabras reservadas del lenguaje.

Buenas prácticas

Una variable debe describir claramente la información que almacena.

Mal ejemplo

```
a = 25  
b = 7  
c = a * b
```

Buen ejemplo

```
base = 25  
altura = 7  
area = base * altura
```

El segundo programa puede entenderse incluso sin comentarios.

Convención de nombres (snake_case)

Python recomienda utilizar **snake_case**.

Consiste en escribir todas las palabras en minúsculas separadas por guiones bajos.

Ejemplos

```
nombre_completo  
precio_final  
cantidad_alumnos  
nota_promedio
```

No es obligatorio, pero sí una buena práctica ampliamente utilizada.

Ejercicio resuelto

Analicemos el siguiente programa.

```
numero = 10  
numero = numero + 5  
print(numero)
```

Paso 1

```
numero  
10
```

Paso 2

Python calcula primero la expresión de la derecha.

```
10 + 5
```

Resultado

```
15
```

Luego reemplaza el contenido de la variable.


```
numero
```

```
15
```

Salida

```
15
```

4. Tipos de datos

¿Qué es un tipo de dato?

Cuando una persona escribe información, puede almacenar distintos tipos de datos.

Por ejemplo:

Información	Ejemplo
Un nombre	Juan
Una edad	17
Una nota	8.75
Una respuesta de verdadero o falso	True

Para una computadora ocurre exactamente lo mismo.

Un **tipo de dato** indica qué clase de información se almacenará en una variable y qué operaciones pueden realizarse con ella.

Cada vez que Python crea una variable, identifica automáticamente el tipo de dato correspondiente.

Por ejemplo:

```
edad = 17
```

Python reconoce que **17** es un número entero.

En cambio:

```
nombre = "Juan"
```

Python reconoce que se trata de una cadena de texto.

Esta característica se conoce como **tipado dinámico**, ya que el programador no necesita indicar explícitamente el tipo de dato de cada variable.

Los tipos de datos principales en Python

Durante este curso utilizaremos principalmente cinco tipos de datos.

Tipo	Nombre en Python	Ejemplo
Entero	<code>int</code>	15
Decimal	<code>float</code>	3.14
Booleano	<code>bool</code>	True
Cadena	<code>str</code>	"Hola"
Carácter	<code>str</code>	"A"

Importante

A diferencia de otros lenguajes como Java, Python **no posee un tipo de dato específico para los caracteres**. Un carácter se representa simplemente como una cadena (`str`) de longitud 1.

El tipo entero (`int`)

Los enteros representan números **sin parte decimal**.

Ejemplos:

```
edad = 16
cantidad_alumnos = 32
temperatura = -5
```

Todos estos valores pertenecen al tipo `int`.

Operaciones con enteros

Python permite realizar las operaciones matemáticas habituales.

```
a = 20
b = 5

print(a + b)
print(a - b)
```

```
print(a * b)
print(a / b)
```

Salida

```
25
15
100
4.0
```

Observemos que la división (/) devuelve un número decimal, incluso cuando el resultado es exacto.

División entera

Python posee otro operador para realizar divisiones enteras.

```
a = 17
b = 5

print(a // b)
```

Salida

```
3
```

La parte decimal se descarta.

Resto de una división (módulo)

El operador % devuelve el resto de una división.

```
print(17 % 5)
```

Salida

```
2
```

Porque

```
17 = 5 × 3 + 2
```

Este operador es muy útil para determinar, por ejemplo, si un número es par o impar.

```
numero = 18  
print(numero % 2)
```

Resultado

```
0
```

Como el resto es cero, el número es par.

Potencia

Para calcular potencias se utiliza el operador `**`.

```
print(2 ** 5)
```

Resultado

```
32
```

Prioridad de operadores

Python sigue el mismo orden de prioridad que en Matemática.

1. Paréntesis
2. Potencias
3. Multiplicación, división y módulo
4. Suma y resta

Ejemplo

```
resultado = 3 + 4 * 2
```

Python realiza primero la multiplicación.

```
3 + 8
```

```
11
```

El tipo decimal (`float`)

Los números con parte decimal pertenecen al tipo `float`.

Ejemplos

```
altura = 1.78
```

```
promedio = 8.50
```

```
precio = 3499.99
```

Importante

En Python el separador decimal es el **punto**, no la coma.

Correcto

```
pi = 3.1416
```

Incorrecto

```
pi = 3,1416
```

Operaciones con decimales

```
precio = 2500.50
```

```
iva = precio * 0.21
```

```
print(iva)
```

Salida

```
525.105
```

El tipo booleano (`bool`)

Un dato booleano solamente puede tomar dos valores.

```
True
False
```

Se utilizan principalmente para representar condiciones.

Ejemplo

```
es_mayor = True
tiene_permiso = False
```

Expresiones booleanas

Las comparaciones generan valores booleanos.

```
print(8 > 5)
```

Resultado

```
True
```

```
print(10 == 7)
```

Resultado

```
False
```

Operadores relacionales

Operador	Significado
<code>==</code>	Igual
<code>!=</code>	Distinto
<code>></code>	Mayor

Operador	Significado
<	Menor
>=	Mayor o igual
<=	Menor o igual

Ejemplos

```
print(8 >= 8)
print(15 != 20)
print(3 < 1)
```

Resultado

```
True
True
False
```

El tipo cadena (str)

Las cadenas almacenan texto.

Siempre deben escribirse entre comillas.

```
nombre = "Juan"
apellido = 'Pérez'
```

Python acepta tanto comillas simples como dobles.

Concatenación

Concatenar significa unir cadenas.

```
nombre = "Ana"
apellido = "López"
```

```
print(nombre + " " + apellido)
```

Resultado

```
Ana López
```

Repetición de cadenas

También pueden multiplicarse.

```
print("Hola " * 3)
```

Resultado

```
Hola Hola Hola
```

Longitud de una cadena

La función `len()` devuelve la cantidad de caracteres.

```
nombre = "Python"  
print(len(nombre))
```

Resultado

```
6
```

El concepto de carácter

Aunque en Python no existe un tipo `char`, una cadena puede contener un solo carácter.

```
inicial = "A"  
respuesta = "S"
```



```
letra = "z"
```

Todas son variables de tipo `str`.

Entrada de datos

Hasta ahora escribíamos los valores directamente.

```
edad = 17
```

Sin embargo, la mayoría de los programas necesitan que sea el usuario quien ingrese la información.

Para ello se utiliza la función `input()`.

```
nombre = input("Ingrese su nombre: ")
```

Cuando el programa llega a esta línea:

1. Muestra el mensaje.
 2. Espera que el usuario escriba.
 3. Guarda la respuesta.
-

Ejemplo

```
nombre = input("Nombre: ")  
print("Hola", nombre)
```

Si el usuario escribe

```
Martina
```

La salida será

```
Hola Martina
```

Un detalle muy importante

La función `input()` **siempre devuelve una cadena (`str`)**, incluso si el usuario escribe un número.

Por ejemplo:

```
edad = input("Edad: ")
```

Si el usuario escribe

```
16
```

La variable `edad` contiene

```
"16"
```

No contiene el número entero 16.

Esto suele ser una de las primeras causas de error para quienes comienzan a programar.

Conversión de tipos (Casting)

En muchas ocasiones es necesario convertir un dato de un tipo a otro.

Python proporciona varias funciones para ello.

Función	Conversión
<code>int()</code>	Convierte a entero
<code>float()</code>	Convierte a decimal
<code>str()</code>	Convierte a cadena
<code>bool()</code>	Convierte a booleano (según las reglas de Python)

Convertir una cadena a entero

```
edad = input("Edad: ")  
edad = int(edad)  
print(edad + 1)
```

Si el usuario ingresa:

16

El resultado será:

17

Forma abreviada

Es habitual combinar ambas funciones en una sola línea.

```
edad = int(input("Edad: "))
```

Convertir a decimal

```
altura = float(input("Altura: "))
```

Convertir un número a cadena

```
numero = 25  
texto = str(numero)  
print(texto)
```

Buenas prácticas

- Utilizar nombres de variables descriptivos.
 - Elegir el tipo de dato adecuado para cada situación.
 - Convertir los datos ingresados mediante `input()` antes de realizar operaciones matemáticas.
 - Evitar mezclar cadenas y números sin realizar conversiones explícitas.
-

Ejercicios de práctica

1. Crear un programa que solicite el nombre y la edad de una persona y muestre un mensaje de bienvenida.

2. Solicitar dos números enteros y mostrar:

- La suma.
- La resta.
- El producto.
- La división.
- La división entera.
- El resto de la división.

3. Solicitar el radio de un círculo y calcular su área.

4. Pedir la cantidad de horas trabajadas y el valor por hora. Calcular el salario total.

5. Solicitar una temperatura en grados Celsius y convertirla a Fahrenheit.

6. Pedir dos números y mostrar si el primero es mayor que el segundo utilizando una expresión booleana.

7. Solicitar un número entero e indicar si es par o impar utilizando el operador %.

5. Alcance de las variables (Scope)

¿Qué es el alcance de una variable?

El **alcance** (en inglés, *scope*) es la región del programa donde una variable puede utilizarse.

En otras palabras, determina **desde qué partes del programa una variable es visible y accesible**.

Dependiendo del lugar donde se crea una variable, esta puede ser:

- **Variable local**
- **Variable global**

Comprender esta diferencia es fundamental para escribir programas bien organizados y evitar errores difíciles de detectar.

¿Por qué existen variables locales?

Imaginemos una escuela.

Cada aula tiene una pizarra.

Lo que escribe el profesor en la pizarra del Aula 1 solamente puede verse dentro de esa aula.

Los alumnos del Aula 2 no pueden verla.

Con las variables locales ocurre exactamente lo mismo.

Una variable creada dentro de una función solo existe dentro de esa función.

¿Qué es una función?

Antes de continuar necesitamos introducir un concepto que utilizaremos durante todo el curso.

Una **función** es un bloque de instrucciones que realiza una tarea específica.

Ejemplo:

```
def saludar():  
    print("Hola")
```

Todavía no profundizaremos en funciones (eso se verá en B2.3.4), pero las utilizaremos para comprender el alcance de las variables.

Variable local

Definición

Una variable local es aquella que se crea **dentro de una función**.

Solo puede utilizarse dentro de esa función.

Cuando la función termina de ejecutarse, la variable desaparece.

Ejemplo 1

```
def saludar():  
    nombre = "Martina"  
    print(nombre)  
saludar()
```

Salida

Martina

Todo funciona correctamente.

¿Qué ocurre en memoria?

Cuando comienza la función:

```
saludar()
```

```
+-----+
| nombre |
| "Martina" |
+-----+
```

Al finalizar la función:

La memoria utilizada por la variable se libera.

nombre deja de existir.

Intentar acceder a una variable local

Analicemos el siguiente programa.

```
def saludar():
    nombre = "Martina"
saludar()
print(nombre)
```

Resultado

NameError

name 'nombre' is not defined

¿Por qué ocurre?

Porque la variable **nombre** fue creada dentro de la función.

Cuando la función terminó, dejó de existir.

Tabla de seguimiento

Línea	Acción	Variables
1	Se crea la función	—
5	Se ejecuta la función	—
3	Se crea nombre	nombre = "Martina"
4	Finaliza la función	nombre desaparece
7	Se intenta imprimir nombre	Error

Otro ejemplo

```
def calcular():  
    numero = 20  
    print(numero)  
calcular()
```

Salida

```
20
```

Pero

```
def calcular():  
    numero = 20  
calcular()  
print(numero)
```

produce nuevamente

```
NameError
```

Ventajas de las variables locales

Las variables locales presentan varias ventajas:

- Ocupan memoria únicamente mientras la función se ejecuta.
- Evitan modificar accidentalmente datos utilizados por otras funciones.
- Facilitan la lectura del código.
- Hacen que los programas sean más seguros.

Por estas razones, siempre que sea posible se recomienda utilizar variables locales.

Variable global

Definición

Una variable global es aquella que se declara **fuera de cualquier función**.

Puede utilizarse desde cualquier parte del programa.

Ejemplo

```
nombre = "Carlos"

def mostrar():
    print(nombre)

mostrar()
```

Salida

Carlos

La variable existe antes de ejecutar la función.

Visualización

Programa

```
+-----+
| nombre |
| Carlos |
```


+-----+

↓

Función mostrar()

Puede acceder a nombre.

Más de una función

```
contador = 10

def uno():
    print(contador)

def dos():
    print(contador)

uno()
dos()
```

Salida

10

10

Ambas funciones utilizan la misma variable.

¿Puede modificarse una variable global?

Veamos este programa.

```
contador = 10

def aumentar():
    contador = contador + 1

aumentar()
```

¿Qué sucede?

Resultado

UnboundLocalError

¿Por qué ocurre?

Python interpreta que:

```
contador = contador + 1
```

significa

"Voy a crear una variable local llamada contador."

Pero antes intenta leerla.

Como todavía no existe...

produce un error.

La palabra reservada global

Si realmente queremos modificar una variable global desde una función debemos indicárselo explícitamente a Python.

Para ello se utiliza la palabra reservada:

```
global
```

Ejemplo

```
contador = 10

def aumentar():
    global contador
    contador = contador + 1

aumentar()
```

```
print(contador)
```

Salida

```
11
```

¿Qué hace global?

La línea

```
global contador
```

le indica a Python:

"No crees una variable nueva."

"Utiliza la variable global que ya existe."

Tabla de seguimiento

Estado inicial

```
contador
```

```
10
```

Se ejecuta la función.

```
contador = contador + 1
```

↓

```
11
```

Después

```
contador
```

```
11
```

¿Siempre debemos usar variables globales?

No.

De hecho, en programación profesional se intenta evitarlas siempre que sea posible.

¿Por qué?

Imaginemos un programa de 10.000 líneas.

Si cualquier función puede modificar una variable global, será muy difícil descubrir quién cambió su valor.

Por eso las variables globales pueden provocar errores difíciles de encontrar.

Buenas prácticas

- ✓ Utilizar variables locales siempre que sea posible.
 - ✓ Reservar las variables globales únicamente para información compartida por todo el programa.
 - ✓ Evitar modificar variables globales desde muchas funciones diferentes.
 - ✓ Elegir nombres claros y descriptivos.
-

Comparación

Variable Local	Variable Global
Se crea dentro de una función	Se crea fuera de las funciones
Solo existe dentro de esa función	Existe durante toda la ejecución del programa
Se elimina al finalizar la función	Permanece en memoria
Es más segura	Puede ser modificada desde distintos lugares
Se recomienda utilizarla	Debe utilizarse con cuidado

Actividad integradora

Desarrollar un programa que simule un contador de visitas a una página web.

1. Crear una variable global llamada `visitas` inicializada en `0`.
 2. Crear una función `registrar_visita()` que incremente el contador utilizando la palabra reservada `global`.
 3. Crear una función `mostrar_visitas()` que muestre el número actual de visitas.
 4. Llamar varias veces a `registrar_visita()` y comprobar que el contador aumenta correctamente.
-

B2.1.2 – Manipulación de cadenas (Strings)

1. ¿Qué es una cadena de caracteres?

Una **cadena de caracteres** (*string*) es una secuencia ordenada de caracteres utilizada para almacenar texto.

Una cadena puede contener:

- Letras.
- Números.
- Espacios.
- Símbolos.
- Signos de puntuación.

Por ejemplo:

```
nombre = "María"
direccion = "Av. Belgrano 1250"
mensaje = "Hola Mundo"
```

Todas estas variables son de tipo `str`.

¿Por qué una cadena es una secuencia?

La palabra **secuencia** significa que cada carácter ocupa una posición determinada.

Por ejemplo:

```
P Y T H O N
```

Python numera automáticamente cada posición comenzando desde cero.

Posición

0	1	2	3	4	5
P	Y	T	H	O	N

Esto permitirá acceder a cualquier carácter individual.

2. Índices

Cada carácter posee un índice.

Supongamos:

```
palabra = "Python"
```

Visualmente:

Índice

0	1	2	3	4	5
P	y	t	h	o	n

Para acceder a un carácter se utilizan corchetes.

```
print(palabra[0])
```

Salida

```
P
```

```
print(palabra[3])
```

Salida

```
h
```

```
print(palabra[5])
```

Salida

```
n
```

¿Qué ocurre si el índice no existe?

```
palabra = "Python"  
print(palabra[10])
```

Resultado

```
IndexError  
string index out of range
```

Porque la palabra solamente posee seis caracteres.

Índices negativos

Python también permite comenzar a contar desde el final.

```
P   y   t   h   o   n  
-6 -5 -4 -3 -2 -1
```

Ejemplo

```
print(palabra[-1])
```

Resultado

```
n
```

```
print(palabra[-2])
```

Resultado

```
o
```

Esto resulta muy útil cuando se desea acceder al último carácter sin conocer la longitud de la cadena.

3. Longitud de una cadena

La función `len()` devuelve la cantidad de caracteres.

```
palabra = "Programación"  
print(len(palabra))
```

Resultado

```
12
```

Importante: Los espacios también cuentan como caracteres.

Ejemplo:

```
texto = "Hola Mundo"  
print(len(texto))
```

Resultado

```
10
```

(El espacio entre "Hola" y "Mundo" cuenta como un carácter).

4. Recorrer una cadena

Una cadena puede recorrerse igual que una lista.

Utilizando for

```
nombre = "Ana"
for letra in nombre:
    print(letra)
```

Salida

```
A
n
a
```

Utilizando índices

```
nombre = "Ana"
for i in range(len(nombre)):
    print(nombre[i])
```

Salida

```
A
n
a
```

5. Subcadenas (Slicing)

Una de las características más potentes de Python es la posibilidad de extraer partes de una cadena.

La sintaxis es:

```
cadena[inicio:fin]
```

Python toma el carácter indicado por **inicio** y llega hasta **fin**, **sin incluirlo**.

Ejemplo

```
palabra = "Programación"  
print(palabra[0:4])
```

Resultado

Prog

¿Por qué?

Porque toma:

```
0 1 2 3  
P r o g
```

El índice 4 no se incluye.

Otro ejemplo

```
print(palabra[4:8])
```

Resultado

rama

Omitiendo el inicio

Si se omite el primer número, Python comienza desde el principio.

```
print(palabra[:6])
```

Resultado

Progra

Omitiendo el final

```
print(palabra[6:])
```

Resultado

```
mación
```

Copiar una cadena completa

```
copia = palabra[:]
```

Se obtiene una copia completa de la cadena.

Salto

También puede indicarse un tercer valor.

```
cadena[inicio:fin:paso]
```

Ejemplo

```
texto = "ABCDEFGHI"  
print(texto[0:9:2])
```

Resultado

```
ACEGI
```

Porque toma un carácter y salta uno.

Invertir una cadena

Un caso muy utilizado es:

```
texto[::-1]
```

Ejemplo

```
texto = "Python"  
print(texto[::-1])
```

Resultado

nohtyP

6. Concatenación

Concatenar significa unir cadenas.

```
nombre = "Juan"  
apellido = "Pérez"  
print(nombre + " " + apellido)
```

Salida

Juan Pérez

También pueden concatenarse variables.

```
frase = "Hola"  
frase = frase + " Mundo"  
print(frase)
```

Resultado

Hola Mundo

7. Repetición

El operador `*` repite una cadena.

```
print("Python " * 3)
```

Resultado

```
Python Python Python
```

8. Métodos de las cadenas

Los métodos son funciones asociadas a un objeto.

Se escriben utilizando la siguiente sintaxis:

```
cadena.metodo()
```

upper()

Convierte todos los caracteres a mayúsculas.

```
nombre = "Martina"  
print(nombre.upper())
```

Resultado

```
MARTINA
```

lower()

Convierte todos los caracteres a minúsculas.

```
print(nombre.lower())
```

Resultado

```
martina
```

capitalize()

Convierte únicamente la primera letra en mayúscula.

```
print(nombre.capitalize())
```

Resultado

```
Martina
```

title()

Convierte en mayúscula la primera letra de cada palabra.

```
titulo = "introducción a python"  
print(titulo.title())
```

Resultado

```
Introducción A Python
```

strip()

Elimina espacios al comienzo y al final.

```
texto = "  Python  "  
print(texto.strip())
```

Resultado

```
Python
```

replace()

Reemplaza una parte de la cadena.

```
frase = "Me gusta Java"  
print(frase.replace("Java", "Python"))
```

Resultado

```
Me gusta Python
```

find()

Busca una subcadena y devuelve su posición.

```
texto = "Programación"  
print(texto.find("gra"))
```

Resultado

```
3
```

Si no encuentra la subcadena devuelve -1.

startswith()

Determina si una cadena comienza con un texto determinado.

```
archivo = "datos.txt"  
print(archivo.startswith("datos"))
```

Resultado

```
True
```

endswith()

Permite verificar cómo termina una cadena.

```
archivo = "datos.txt"  
print(archivo.endswith(".txt"))
```

Resultado

```
True
```

Las cadenas son inmutables

Un aspecto muy importante de Python es que las cadenas son **inmutables**.

Esto significa que **no pueden modificarse directamente**.

Ejemplo incorrecto:

```
palabra = "Python"
palabra[0] = "J"
```

Resultado

```
TypeError
```

La forma correcta consiste en crear una nueva cadena.

```
palabra = "Python"
palabra = "J" + palabra[1:]
print(palabra)
```

Resultado

```
Jython
```

Ejercicio resuelto

Desarrollar un programa que solicite el nombre completo de una persona y muestre:

- La cantidad de caracteres.
- El nombre en mayúsculas.
- El nombre en minúsculas.
- La primera letra.
- La última letra.

Solución

```
nombre = input("Ingrese su nombre completo: ")
```



```
print("Cantidad de caracteres:", len(nombre))
print("Mayúsculas:", nombre.upper())
print("Minúsculas:", nombre.lower())
print("Primera letra:", nombre[0])
print("Última letra:", nombre[-1])
```

Ejercicios de práctica

1. Solicitar una palabra y mostrar cada uno de sus caracteres en una línea diferente.
 2. Pedir un nombre completo e imprimir únicamente las tres primeras letras.
 3. Mostrar los últimos cuatro caracteres de una cadena ingresada por el usuario.
 4. Solicitar una frase y mostrarla completamente en mayúsculas y luego en minúsculas.
 5. Pedir una oración y reemplazar todas las apariciones de la palabra "Python" por "Java".
 6. Solicitar un correo electrónico y verificar si termina con "@gmail.com".
 7. Pedir una palabra y mostrarla invertida utilizando *slicing*.
 8. Solicitar una oración y contar cuántos caracteres tiene, incluyendo los espacios.
 9. Pedir un nombre y mostrar únicamente la inicial en mayúscula seguida de un punto (por ejemplo, "Carlos" → "C. ").
 10. Escribir un programa que solicite una frase e indique si contiene la palabra "IB" utilizando el método `find()`.
-

9. Aplicaciones prácticas de manipulación de cadenas

Hasta el momento hemos aprendido a:

- acceder a un carácter mediante índices;
- extraer subcadenas (*slicing*);
- concatenar cadenas;
- reemplazar texto;
- buscar texto;
- convertir entre mayúsculas y minúsculas.

Sin embargo, en un programa real estas operaciones rara vez se utilizan de forma aislada. Lo habitual es combinar varias de ellas para resolver un problema.

En esta sección veremos algunos ejemplos representativos.

Problema 1 – Obtener las iniciales de una persona

Supongamos que el usuario ingresa:

Juan Pérez

Queremos mostrar:

J.P.

Análisis

Sabemos que:

- el primer nombre termina en un espacio;
 - la primera letra del nombre es la posición 0;
 - la primera letra del apellido está inmediatamente después del espacio.
-

Solución

```
nombre = input("Nombre completo: ")
espacio = nombre.find(" ")
inicial_nombre = nombre[0]
inicial_apellido = nombre[espacio + 1]
print(inicial_nombre + "." + inicial_apellido + ".")
```

Seguimiento

Si el usuario escribe

Juan Pérez

La variable

```
nombre
```

```
Juan Pérez
```

Después

```
espacio
```

```
4
```

Porque el espacio está en la posición 4.

Luego

```
nombre[0]
```

```
J
```

y

```
nombre[5]
```

```
P
```

Resultado

```
J.P.
```

Problema 2 – Obtener el dominio de un correo electrónico

Supongamos:

```
franco@gmail.com
```

Queremos obtener únicamente:

```
gmail.com
```

Análisis

Debemos localizar el símbolo

```
@
```

Luego extraer todo lo que se encuentra después.

Solución

```
correo = input("Correo: ")
arroba = correo.find("@")
dominio = correo[arroba + 1:]
print(dominio)
```

Ejemplo

Entrada

```
ana@hotmail.com
```

Salida

```
hotmail.com
```

Problema 3 – Obtener el nombre de usuario

Ahora queremos hacer exactamente lo contrario.

De

```
franco@gmail.com
```

obtener

```
franco
```

Solución

```
correo = input("Correo: ")
arroba = correo.find("@")
usuario = correo[:arroba]
print(usuario)
```

Problema 4 – Ocultar una contraseña

Supongamos que el usuario escribe

Python123

Queremos mostrar

La cantidad de asteriscos debe coincidir con la longitud de la contraseña.

Solución

```
password = input("Contraseña: ")
print("*" * len(password))
```

Problema 5 – Verificar extensión de archivo

Entrada

foto.jpg

Salida

Es una imagen JPG.

```
archivo = input("Archivo: ")  
  
if archivo.endswith(".jpg"):  
    print("Es una imagen JPG")  
else:  
    print("No es una imagen JPG")
```

Problema 6 – Capitalizar correctamente un nombre

Muchas personas escriben:

mArTiN

Queremos mostrar

Martin

```
nombre = input("Nombre: ")  
  
print(nombre.capitalize())
```

Problema 7 – Eliminar espacios innecesarios

Entrada

Python

Salida

Python

```
texto = input("Ingrese texto: ")  
  
texto = texto.strip()
```

```
print(texto)
```

Problema 8 – Contar cuántas veces aparece una letra

Queremos saber cuántas veces aparece la letra **a**.

Python posee el método

```
count()
```

Ejemplo

```
texto = "banana"
print(texto.count("a"))
```

Resultado

```
3
```

También podemos contar palabras.

```
frase = "Python es un lenguaje. Python es sencillo."
print(frase.count("Python"))
```

Resultado

```
2
```

Problema 9 – Saber si una palabra está contenida en otra

Existe una forma muy sencilla.

```
if "Python" in texto:
```

Ejemplo

```
texto = input("Ingrese una oración: ")

if "Python" in texto:
    print("La palabra aparece.")
else:
    print("No aparece.")
```

También funciona con letras.

```
if "@" in correo:
```

Problema 10 – Reemplazar varias palabras

```
texto = "Me gusta Java"
texto = texto.replace("Java", "Python")
print(texto)
```

Resultado

```
Me gusta Python
```

Encadenando métodos

Los métodos pueden combinarse.

Ejemplo

```
nombre = input("Nombre: ")
print(nombre.strip().upper())
```

Si el usuario escribe

franco

Python realiza:

```
strip()
```

↓

franco

Luego

```
upper()
```

↓

FRANCO

Otro ejemplo

```
correo = input("Correo: ")
correo = correo.strip().lower()
print(correo)
```

Esto permite normalizar la información antes de almacenarla.

Ejercicio resuelto

Desarrollar un programa que solicite un correo electrónico y muestre:

- Usuario.
- Dominio.
- Cantidad de caracteres.
- Dominio en mayúsculas.

Solución

```
correo = input("Correo electrónico: ")
arroba = correo.find("@")
```

```
usuario = correo[:arroba]
dominio = correo[arroba + 1:]
print("Usuario:", usuario)
print("Dominio:", dominio)
print("Cantidad de caracteres:", len(correo))
print("Dominio en mayúsculas:", dominio.upper())
```

Mini proyecto integrador

Analizador de nombres

Desarrollar un programa que solicite el nombre completo de una persona y muestre:

- Nombre completo.
- Cantidad de caracteres.
- Cantidad de palabras.
- Nombre en mayúsculas.
- Nombre en minúsculas.
- Primera letra.
- Última letra.
- Iniciales.
- Nombre invertido.
- Si contiene la palabra "María".

Ejemplo

Entrada:

María Fernanda López

Salida:

Nombre: María Fernanda López

Cantidad de caracteres: 21

Cantidad de palabras: 3

Mayúsculas: MARÍA FERNANDA LÓPEZ

Minúsculas: maría fernanda lópez

Primera letra: M

Última letra: z

Iniciales: M.F.L.

Invertido: zepóL adnanreF aíraM

¿Contiene "María"?: Sí

B2.1.3 – Manejo de excepciones

1. Los errores en programación

Cuando un programa es ejecutado, no siempre se comporta como esperamos.

En muchas ocasiones pueden producirse situaciones inesperadas que impiden que el programa continúe funcionando correctamente.

Por ejemplo:

- El usuario escribe texto cuando se esperaba un número.
- Se intenta abrir un archivo que no existe.
- Se realiza una división entre cero.
- Se intenta acceder a una posición inexistente de una lista.
- El programa contiene una instrucción escrita incorrectamente.

Todos estos casos producen **errores**.

Un buen programador no solamente escribe programas que funcionan cuando todo sale bien, sino también programas capaces de reaccionar correctamente cuando ocurre un problema.

¿Qué ocurre si no controlamos los errores?

Observemos el siguiente programa.

```
edad = int(input("Edad: "))  
print("El año próximo tendrá", edad + 1)
```

Si el usuario escribe

```
16
```

todo funciona correctamente.

Salida

```
El año próximo tendrá 17
```

Pero supongamos que el usuario escribe

```
dieciséis
```

Python responde

```
ValueError:  
invalid literal for int()
```

El programa termina inmediatamente.

No continúa ejecutándose.

Esto genera una mala experiencia para el usuario.

¿Qué es una excepción?

Una **excepción** es un evento que ocurre durante la ejecución de un programa e interrumpe su flujo normal.

Cuando Python detecta una situación que no puede resolver automáticamente, genera una excepción.

Si el programador no la controla, el programa finaliza mostrando un mensaje de error.

Tipos de errores

En programación existen tres grandes categorías de errores.

- Errores de sintaxis.
- Errores de ejecución.
- Errores lógicos.

Comprender esta diferencia es muy importante.

No todos los errores se solucionan de la misma manera.

2. Errores de sintaxis

Son errores que impiden que el programa pueda comenzar a ejecutarse.

Generalmente ocurren porque el código no respeta las reglas del lenguaje.

Ejemplo

```
print("Hola)
```

Resultado

```
SyntaxError
```

La comilla de cierre falta.

Python ni siquiera puede ejecutar el programa.

Otro ejemplo

```
if edad > 18
    print("Mayor")
```

Resultado

```
SyntaxError
```

Falta el carácter

```
:
```

Características

Los errores sintácticos:

- Se detectan antes de comenzar la ejecución.
 - Son fáciles de localizar.
 - Generalmente indican la línea exacta donde se produjo el problema.
-

3. Errores de ejecución

También se denominan **errores en tiempo de ejecución** (*runtime errors*).

El programa comienza correctamente.

Sin embargo, durante la ejecución ocurre una situación inesperada.

Ejemplo

```
numero = int(input("Número: "))
```

Si el usuario escribe

```
hola
```

Python produce

```
ValueError
```

Otro ejemplo

```
print(20 / 0)
```

Resultado

```
ZeroDivisionError
```

Otro ejemplo

```
lista = [4,8,2]
print(lista[10])
```

Resultado

IndexError

Otro ejemplo

```
archivo = open("datos.txt")
```

Si el archivo no existe

FileNotFoundError

Características

Los errores de ejecución:

- aparecen mientras el programa está funcionando;
 - dependen muchas veces de la información ingresada por el usuario;
 - pueden controlarse mediante excepciones.
-

4. Errores lógicos

Son los más difíciles de detectar.

El programa funciona.

No aparece ningún mensaje de error.

Pero el resultado es incorrecto.

Ejemplo

Queremos calcular el promedio.

```
nota1 = 8
nota2 = 6
```

```
promedio = nota1 + nota2 / 2
print(promedio)
```

Python muestra

```
11
```

No existe ningún error.

Sin embargo, el promedio correcto debería ser

```
7
```

El problema es que olvidamos utilizar paréntesis.

La solución correcta es

```
promedio = (nota1 + nota2) / 2
```

Otro ejemplo

```
edad = 18
if edad > 18:
    print("Mayor de edad")
```

Si el usuario tiene exactamente 18 años

no aparece ningún mensaje.

El programa funciona.

Pero la condición elegida fue incorrecta.

Características

Los errores lógicos:

- no producen excepciones;
 - son los más difíciles de encontrar;
 - requieren analizar cuidadosamente el algoritmo.
-

Comparación

Tipo	¿El programa se ejecuta?	¿Produce mensaje de error?
Sintaxis	No	Sí
Ejecución	Comienza pero se detiene	Sí
Lógico	Sí	No

¿Cómo evitar que un programa termine?

Python incorpora un mecanismo denominado **manejo de excepciones**.

Su objetivo consiste en controlar los errores previsibles para que el programa pueda continuar ejecutándose.

La estructura básica es:

```
try:
    # Código que puede producir un error
except:
    # Código que se ejecuta si ocurre un error
```

5. La instrucción `try`

La palabra reservada `try` significa literalmente:

"Intentar ejecutar este bloque de código."

Si todo funciona correctamente, Python continúa normalmente.

Si aparece una excepción, abandona inmediatamente el bloque `try` y busca un bloque `except`.

Ejemplo

```
try:
    numero = int(input("Número: "))
    print(numero)
```

```
except:  
    print("Debe ingresar un número.")
```

Caso 1

Entrada

25

Salida

25

Caso 2

Entrada

Hola

Salida

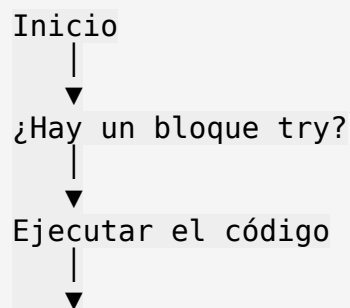
Debe ingresar un número.

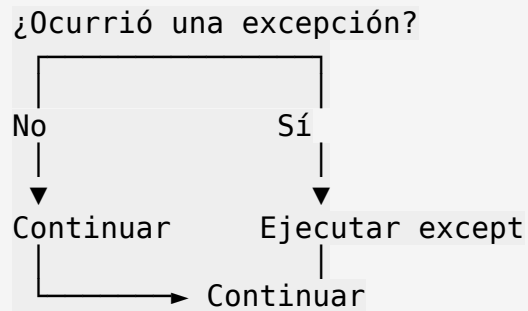
En este caso el programa **no finaliza abruptamente**.

Continúa funcionando gracias al bloque `except`.

Flujo de ejecución

Cuando Python encuentra un bloque `try`, sigue este proceso:





¿Qué ocurre si no hay error?

Observemos el siguiente ejemplo.

```
try:
    numero = int(input("Número: "))
    print(numero)
except:
    print("Error")
```

Si el usuario escribe

50

Python **ignora completamente** el bloque `except`.

Solamente se ejecuta el código del `try`.

Buenas prácticas

Un bloque `try` debe contener **únicamente** las instrucciones que realmente pueden producir una excepción.

Incorrecto:

```
try:
    numero = int(input("Número: "))
    print("Bienvenido")
```

```
print("Programa de ejemplo")
print(numero)
except:
    print("Error")
```

Correcto:

```
try:
    numero = int(input("Número: "))
except:
    print("Debe ingresar un número.")

print("Bienvenido")
print("Programa de ejemplo")
print(numero)
```

De esta forma resulta más sencillo identificar qué instrucción puede fallar y mantener el código organizado.

6. Capturando excepciones específicas

En los ejemplos anteriores utilizamos simplemente:

```
except:
```

Aunque esta forma funciona, **no es recomendable**.

¿Por qué?

Porque captura **cualquier excepción**, incluso aquellas que no esperábamos.

Es una buena práctica indicar qué tipo de excepción deseamos controlar.

La sintaxis general es:

```
try:
    # Código que puede producir una excepción

except TipoDeExcepcion:
    # Código para manejar esa excepción
```

La excepción `ValueError`

Es una de las excepciones más frecuentes cuando trabajamos con `input()`.

Se produce cuando intentamos convertir un dato a un tipo incompatible.

Ejemplo:

```
try:
    edad = int(input("Ingrese su edad: "))
    print("Edad:", edad)
except ValueError:
    print("Error: debe ingresar un número entero.")
```

Entrada válida

18

Salida

Edad: 18

Entrada inválida

dieciocho

Salida

Error: debe ingresar un número entero.

La excepción `ZeroDivisionError`

Se produce cuando intentamos dividir por cero.

```
try:
    numero = int(input("Número: "))
    print(100 / numero)
except ZeroDivisionError:
    print("No es posible dividir por cero.")
```

Entrada

0

Salida

No es posible dividir por cero.

La excepción `IndexError`

Ocurre cuando intentamos acceder a una posición inexistente.

```
numeros = [4, 8, 2]
try:
    print(numeros[5])
except IndexError:
    print("La posición indicada no existe.")
```

Salida

La posición indicada no existe.

La excepción `FileNotFoundError`

Se produce cuando intentamos abrir un archivo que no existe.

```
try:
    archivo = open("datos.txt", "r")
except FileNotFoundError:
    print("No se encontró el archivo.")
```

Más adelante, en el apartado B2.5, veremos con mayor profundidad el trabajo con archivos.

Capturar varias excepciones

Un mismo bloque `try` puede generar distintos tipos de excepciones.

Ejemplo:

```
try:
    numero = int(input("Número: "))
    print(100 / numero)
except ValueError:
    print("Debe ingresar un número.")
except ZeroDivisionError:
    print("No puede dividir por cero.")
```

Caso 1

Entrada

Hola

Salida

Debe ingresar un número.

Caso 2

Entrada

0

Salida

No puede dividir por cero.

Cada excepción es tratada de forma independiente.

7. El bloque `else`

Muchas veces queremos ejecutar un conjunto de instrucciones **únicamente si no ocurrió ningún error**.

Para ello se utiliza `else`.

La estructura es:

```
try:
    ...
except:
    ...
else:
    ...
```

Ejemplo

```
try:
    edad = int(input("Edad: "))
except ValueError:
    print("Edad incorrecta.")
else:
    print("Edad registrada correctamente.")
```

Caso correcto

Entrada

17

Salida

Edad registrada correctamente.

Caso incorrecto

Entrada

diecisiete

Salida

Edad incorrecta.

Observemos que el bloque `else` solamente se ejecuta cuando **no se produjo ninguna excepción**.

¿Por qué utilizar `else`?

Muchas veces el bloque `try` debe contener solamente el código que realmente puede fallar.

Ejemplo incorrecto

```
try:
    edad = int(input("Edad: "))
    print("Bienvenido al sistema.")
    print("Su edad es", edad)
except ValueError:
    print("Error.")
```

Aquí todo el código quedó dentro del `try`.

La versión correcta sería:

```
try:
    edad = int(input("Edad: "))
except ValueError:
    print("Error.")
else:
    print("Bienvenido al sistema.")
```

```
print("Su edad es", edad)
```

El programa resulta más claro y fácil de mantener.

8. El bloque `finally`

Hay ocasiones en las que necesitamos ejecutar una instrucción **siempre**, independientemente de que haya ocurrido un error o no.

Para ello se utiliza `finally`.

La estructura es:

```
try:
    ...
except:
    ...
finally:
    ...
```

Ejemplo

```
try:
    numero = int(input("Número: "))
except ValueError:
    print("Dato inválido.")
finally:
    print("Fin del programa.")
```

Entrada

Hola

Salida

Dato inválido.

Fin del programa.

Entrada

15

Salida

Fin del programa.

`finally` siempre se ejecuta.

¿Cuándo se utiliza `finally`?

Principalmente para liberar recursos.

Por ejemplo:

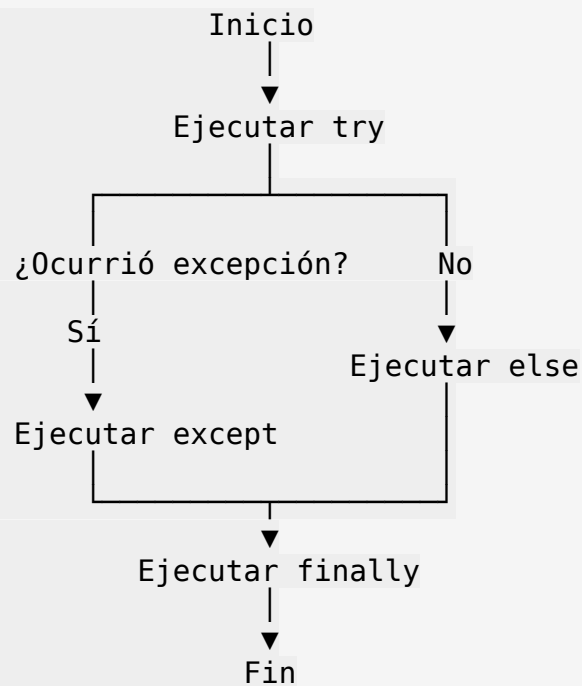
- cerrar archivos;
- cerrar conexiones con bases de datos;
- liberar memoria;
- cerrar conexiones de red.

Ejemplo

```
try:
    archivo = open("datos.txt")
    contenido = archivo.read()
except FileNotFoundError:
    print("Archivo inexistente.")
finally:
    print("Finalizando operación.")
```

Más adelante aprenderemos a cerrar archivos utilizando `close()`.

El flujo completo



Ejemplo completo

```
try:
    numero = int(input("Ingrese un número: "))
    resultado = 100 / numero
except ValueError:
    print("Debe ingresar un número.")
except ZeroDivisionError:
    print("No puede dividir por cero.")
else:
    print("Resultado:", resultado)
finally:
    print("Programa finalizado.")
```

Buenas prácticas

✓ Capturar únicamente las excepciones esperadas

Incorrecto

```
except:
```

Correcto

```
except ValueError:
```

✓ Mantener pequeño el bloque `try`

Solo debe contener el código que realmente puede producir una excepción.

✓ Escribir mensajes comprensibles

Incorrecto

```
Error.
```

Correcto

```
Debe ingresar un número entero.
```

El usuario entiende qué ocurrió y cómo solucionarlo.

✓ No utilizar excepciones para controlar la lógica del programa

Las excepciones deben utilizarse únicamente para situaciones excepcionales.

Por ejemplo, si queremos verificar si un número es mayor que cero, debemos utilizar un `if`, no un `try`.

Ejercicios

Ejercicio 1

Solicitar al usuario un número entero. Si escribe un dato inválido, mostrar un mensaje indicando el error.

Ejercicio 2

Solicitar dos números e imprimir el resultado de la división. Controlar:

- ingreso inválido;
 - división por cero.
-

Ejercicio 3

Crear una lista con cinco números y solicitar una posición. Mostrar el elemento correspondiente controlando el caso en que la posición no exista.

Ejercicio 4

Solicitar el nombre de un archivo e intentar abrirlo. Si el archivo no existe, informar al usuario mediante una excepción.

Ejercicio 5

Escribir un programa que combine `try`, `except`, `else` y `finally` para solicitar un número, calcular su inverso (`1 / número`) y mostrar mensajes apropiados en cada caso.

B2.1.4 – Técnicas de depuración (Debugging)

1. ¿Qué es depurar un programa?

La palabra **depuración** proviene del inglés **debugging**.

Originalmente, un **bug** era cualquier fallo que provocaba que un programa no funcionara correctamente.

Por lo tanto,

Depurar un programa consiste en localizar, comprender y corregir los errores que contiene.

Es importante comprender que:

Encontrar errores es una parte normal del desarrollo de software.

Incluso los programadores profesionales escriben programas con errores.

La diferencia está en que poseen mejores herramientas para encontrarlos.

Depuración vs Manejo de excepciones

Estos conceptos suelen confundirse.

No significan lo mismo.

Depuración	Manejo de excepciones
Busca encontrar errores durante el desarrollo	Permite controlar errores durante la ejecución
La realiza el programador	La ejecuta el propio programa
Corrige el código	Evita que el programa finalice inesperadamente

Ejemplo

Supongamos el siguiente programa.

```
numero = int(input("Número: "))  
print(100 / numero)
```

Si el usuario escribe

```
0
```

Python genera

ZeroDivisionError

Podemos solucionarlo de dos maneras.

Durante el desarrollo

Corregimos el programa.

Esto es depuración.

Durante la ejecución

Utilizamos

```
try:
    ...
except ZeroDivisionError:
    ...
```

Esto es manejo de excepciones.

2. El proceso de depuración

La depuración normalmente sigue cinco pasos.

1. Detectar el error
- ↓
2. Reproducir el problema
- ↓
3. Analizar el programa
- ↓
4. Corregir el error
- ↓
5. Verificar que el problema desapareció

Nunca conviene modificar muchas cosas al mismo tiempo.

Lo recomendable es realizar un cambio pequeño y volver a probar el programa.

3. Utilizar `print()` para depurar

La técnica más sencilla consiste en imprimir el contenido de las variables.

Supongamos el siguiente programa.

```
precio = 150
descuento = 20
total = precio - descuento * 2
print(total)
```

El resultado es

```
110
```

Sin embargo, esperábamos

```
260
```

¿Cómo averiguamos qué ocurre?

Podemos imprimir cada variable.

```
precio = 150
descuento = 20
print("Precio:", precio)
print("Descuento:", descuento)
total = precio - descuento * 2
print("Total:", total)
```

Ahora podemos observar qué valores poseen las variables durante la ejecución.

Imprimiendo expresiones

También es útil imprimir operaciones intermedias.

```
a = 8
b = 4
print(a + b)
print(a * b)
print(a / b)
```

Muchas veces esto permite descubrir rápidamente dónde aparece el problema.

4. Tablas de seguimiento (Trace Tables)

Una de las técnicas más utilizadas en el IB consiste en construir una **tabla de seguimiento**.

Una tabla de seguimiento muestra cómo cambian las variables después de ejecutar cada instrucción.

Ejemplo

```
x = 4
y = 2
x = x + y
y = x * 3
```

La tabla queda:

Línea	x	y
Inicio	—	—
x = 4	4	—
y = 2	4	2
x = x + y	6	2
y = x * 3	6	18

Ejemplo con intercambio

```
a = 5
b = 8
temp = a
a = b
b = temp
```

Tabla

Línea	a	b	temp
Inicio	—	—	—
a = 5	5	—	—
b = 8	5	8	—
temp = a	5	8	5
a = b	8	8	5
b = temp	8	5	5

Las tablas de seguimiento son muy utilizadas en los exámenes del Programa del Diploma.

5. Seguimiento de un bucle

Consideremos el siguiente código.

```
suma = 0
for i in range(1,5):
    suma = suma + i
print(suma)
```

Tabla de seguimiento

Iteración	i	suma
Inicio	—	0
1	1	1
2	2	3
3	3	6

Iteración	i	suma
4	4	10

Salida

10

6. Seguimiento de una condición

```
edad = 16
if edad >= 18:
    categoria = "Mayor"
else:
    categoria = "Menor"
```

Tabla

Variable	Valor
edad	16
Condición	False
categoría	"Menor"

Ejercicios

Ejercicio 1

Realizar la tabla de seguimiento del siguiente programa.

```
x = 3
y = 7
x = x * 2
y = y - x
```

Ejercicio 2

Construir la tabla de seguimiento.

```
total = 0
for i in range(5):
    total += i
```

Ejercicio 3

El siguiente programa contiene un error lógico. Identificarlo y corregirlo.

```
nota1 = 8
nota2 = 6

promedio = nota1 + nota2 / 2
print(promedio)
```

Ejercicio 4

El siguiente programa produce un error. Explicar qué excepción genera y cómo corregirla.

```
lista = [10,20,30]
print(lista[5])
```

CAPÍTULO 2: Estructura de datos

B2.2.1 – Estructuras de selección (if)

1. ¿Qué es una estructura de selección?

Hasta este momento, todos nuestros programas ejecutaban **exactamente las mismas instrucciones**.

Por ejemplo:

```
print("Bienvenido")  
print("Ingrese su nombre")  
print("Programa finalizado")
```

Siempre ocurre lo mismo.

Sin importar quién ejecute el programa.

Pero imaginemos el siguiente problema.

Si el alumno obtiene una nota mayor o igual a 7 deberá aparecer el mensaje:

Aprobado

En caso contrario

Desaprobado

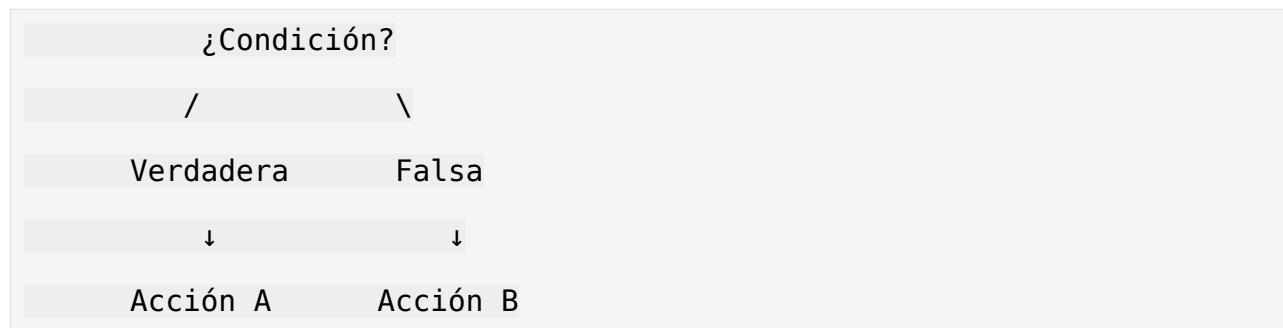
Ahora el programa necesita **elegir**.

Esa elección se realiza mediante una estructura de selección.

¿Qué significa seleccionar?

Seleccionar significa elegir entre distintas alternativas.

Visualmente:



La condición determina cuál será el camino que seguirá el programa.

2. La instrucción `if`

La estructura más sencilla es `if`.

Su sintaxis es:

```
if condición:
    instrucciones
```

Observemos dos detalles muy importantes.

- Después de la condición aparece **dos puntos (:)**.
 - Las instrucciones deben escribirse con **indentación**.
-

¿Qué es la indentación?

La **indentación** es el espacio que aparece al comienzo de una línea.

En Python la indentación **forma parte de la sintaxis del lenguaje**.

No es solamente una cuestión estética.

Ejemplo correcto:

```
edad = 20

if edad >= 18:
    print("Mayor de edad")

print("Fin")
```

Observemos que únicamente la segunda línea pertenece al `if`.

Visualmente:

```
edad = 20
↓
¿edad >= 18?
↓
Sí
↓
print("Mayor de edad")
↓
print("Fin")
```

¿Qué ocurre si la condición es falsa?

Ejemplo

```
edad = 15
if edad >= 18:
    print("Mayor de edad")
print("Programa terminado")
```

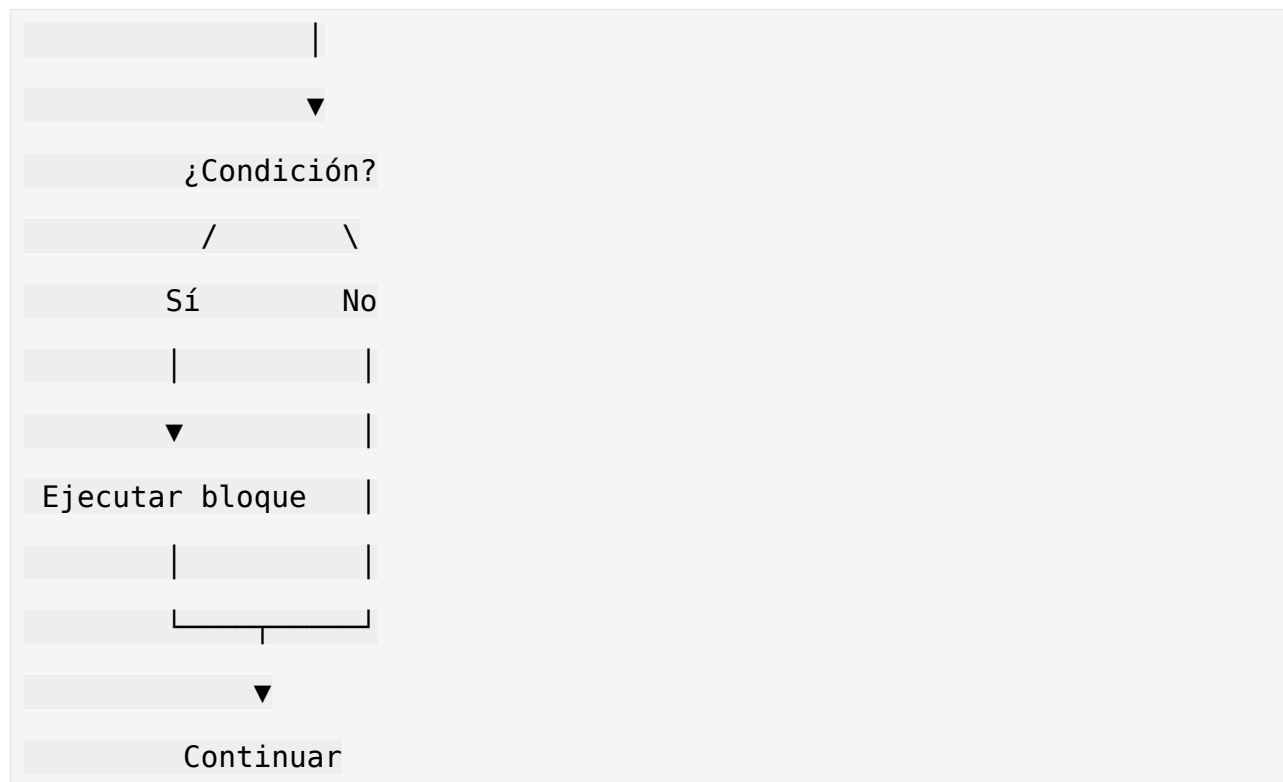
La salida será

```
Programa terminado
```

Simplemente se ignora el bloque del `if`.

Diagrama de flujo

```
Inicio
```

Ejemplo 1

```
temperatura = 35  
  
if temperatura > 30:  
    print("Hace calor")
```

Resultado

Hace calor

Ejemplo 2

```
nota = 9  
  
if nota == 10:  
    print("Excelente")
```

No aparece ningún mensaje.

Porque la condición es falsa.

Ejemplo 3

```
password = input("Contraseña: ")  
  
if password == "python":  
    print("Acceso permitido")
```

Si el usuario escribe

```
python
```

Obtendrá

```
Acceso permitido
```

Si escribe cualquier otra cosa

no ocurrirá nada.

3. Bloques de código

Todo el código con la misma indentación pertenece al mismo bloque.

Ejemplo

```
edad = 20  
  
if edad >= 18:  
    print("Puede votar")  
    print("Es mayor de edad")  
    print("Debe presentar DNI")  
print("Fin")
```

Las tres instrucciones pertenecen al `if`.

Visualmente

```
if
|
├─ print()
├─ print()
└─ print()
↓
Fin
```

4. La estructura `if...else`

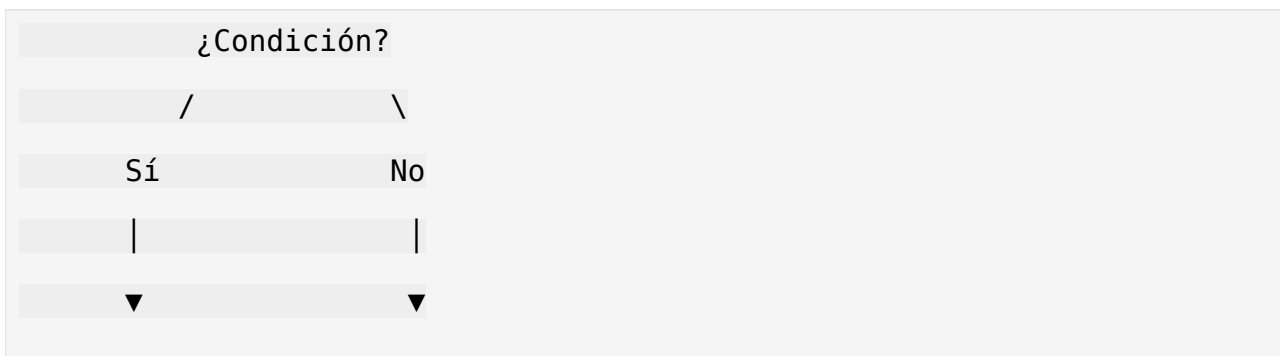
Muchas veces necesitamos ejecutar una acción cuando la condición es verdadera y otra diferente cuando es falsa.

Para ello existe `else`.

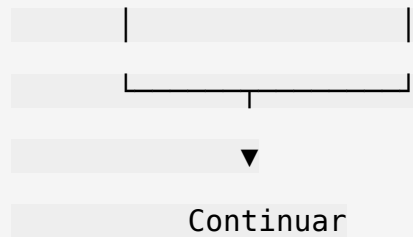
Sintaxis

```
if condición:
    instrucciones
else:
    otras instrucciones
```

Funcionamiento



Bloque if Bloque else



Ejemplo

```
edad = int(input("Edad: "))  
  
if edad >= 18:  
    print("Mayor de edad")  
else:  
    print("Menor de edad")
```

Entrada

20

Salida

Mayor de edad

Entrada

16

Salida

Menor de edad

Tabla de seguimiento

Supongamos

```
edad = 17
```

Variable	Valor
edad	17
edad >=18	False
Bloque ejecutado	else

5. Ejemplo completo

Programa para determinar si un número es par.

```
numero = int(input("Número: "))  
  
if numero % 2 == 0:  
    print("Es par")  
else:  
    print("Es impar")
```

Si el usuario escribe

```
18
```

Python calcula

```
18 % 2
```

```
↓
```

```
0
```

Luego

```
0 == 0
```

```
↓
```

```
True
```

Resultado

Es par

6. Múltiples condiciones (if...elif...else)

Muchas veces existen más de dos posibilidades.

Ejemplo

Una nota puede ser

- Excelente
- Aprobado
- Desaprobado

Aquí aparece

elif

que significa

"Si la condición anterior fue falsa, probar esta otra."

Sintaxis

```
if condición1:
    ...
elif condición2:
    ...
else:
    ...
```

Ejemplo

```
nota = int(input("Nota: "))
if nota >= 9:
```

```
    print("Excelente")
elif nota >= 7:
    print("Aprobado")
else:
    print("Desaprobado")
```

Entrada

10

Salida

Excelente

Entrada

8

Salida

Aprobado

Entrada

5

Salida

Desaprobado

¿Por qué funciona?

Supongamos

nota = 8

Python analiza

```
{nota >= 9?
```

↓

False

Entonces pasa al siguiente.

```
{nota >= 7?
```

↓

True

Como encontró una condición verdadera

deja de evaluar las demás.

Importancia del orden

Observemos este ejemplo.

```
if nota >= 7:  
    print("Aprobado")  
  
elif nota >= 9:  
    print("Excelente")
```

Si

```
nota = 10
```

Python muestra

Aprobado

Nunca llega al segundo `elif`.

Por eso siempre debemos colocar primero las condiciones más específicas.

La versión correcta es

```
if nota >= 9:  
    print("Excelente")
```



```
elif nota >= 7:  
    print("Aprobado")
```

7. Condiciones anidadas

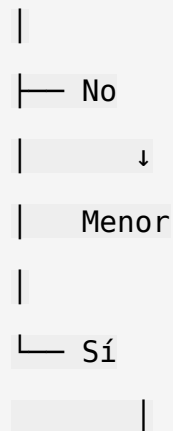
También es posible colocar un `if` dentro de otro.

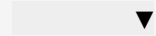
Ejemplo

```
edad = int(input("Edad: "))  
tiene_entrada = input("¿Tiene entrada? (S/N): ")  
if edad >= 18:  
    if tiene_entrada == "S":  
        print("Puede ingresar")  
    else:  
        print("Debe comprar una entrada")  
else:  
    print("Es menor de edad")
```

Diagrama

¿Mayor de edad?





¿Tiene entrada?

|

└ Sí → Ingresa

└ No → Comprar entrada

Ejercicios

Ejercicio 1

Solicitar la edad de una persona e indicar si es mayor o menor de edad.

Ejercicio 2

Pedir un número entero e indicar si es positivo o negativo.

Ejercicio 3

Solicitar dos números e indicar cuál es el mayor.

Ejercicio 4

Solicitar la nota de un alumno y mostrar:

- Excelente (9–10)
 - Aprobado (7–8)
 - Desaprobado (menor que 7)
-

Ejercicio 5

Solicitar un usuario y una contraseña. Si ambos son correctos, mostrar **"Acceso concedido"**; en caso contrario, **"Acceso denegado"**.

Ejercicio 6

Una empresa ofrece descuentos según el monto de una compra:

- Menos de \$10.000: sin descuento.
- Entre \$10.000 y \$49.999: 10% de descuento.
- \$50.000 o más: 20% de descuento.

Desarrollar un programa que solicite el importe de la compra, calcule el descuento correspondiente y muestre el importe final.

B2.2.2 – Condiciones compuestas y operadores lógicos

1. ¿Qué es una condición compuesta?

Hasta ahora trabajamos con condiciones simples.

Ejemplo:

```
edad >= 18
```

o

```
nota >= 7
```

Pero los problemas reales casi nunca dependen de una única condición.

Por ejemplo:

- Un alumno aprueba **si obtiene al menos 7 Y tiene una asistencia mínima del 75%.**
- Un usuario puede ingresar **si es administrador O supervisor.**
- Una persona puede acceder a una promoción **si es cliente frecuente Y realizó una compra superior a determinado importe.**

Estas situaciones requieren combinar varias condiciones.

2. El operador `and`

El operador `and` exige que **todas las condiciones sean verdaderas**.

Ejemplo

```
edad = 20
tiene_dni = True

if edad >= 18 and tiene_dni:
    print("Puede ingresar.")
```

Análisis:

`edad >= 18`

↓

`True`

AND

`tiene_dni`

↓

`True`

Resultado

`True`

Por lo tanto

`Puede ingresar.`

Segundo ejemplo

```
edad = 17
tiene_dni = True
```

Análisis

`False`

AND

True

↓

False

No se ejecuta el bloque del `if`.

Tabla de verdad de `and`

Condición A	Condición B	Resultado
False	False	False
False	True	False
True	False	False
True	True	True

3. El operador `or`

Con `or` alcanza con que **una** de las condiciones sea verdadera.

Ejemplo

```
es_profesor = False
es_directivo = True

if es_profesor or es_directivo:
    print("Puede acceder.")
```

Resultado

Puede acceder.

Tabla de verdad de `or`

Condición A	Condición B	Resultado
False	False	False
False	True	True
True	False	True

Condición A	Condición B	Resultado
True	True	True

4. El operador `not`

`not` invierte el valor lógico.

```
es_admin = False
print(not es_admin)
```

Salida

```
True
```

También puede utilizarse sobre una expresión.

```
edad = 15
if not edad >= 18:
    print("Es menor de edad.")
```

5. Validación de rangos

Uno de los usos más frecuentes de `and` consiste en comprobar si un valor pertenece a un intervalo.

Supongamos que queremos saber si una nota es válida.

Las notas deben estar entre 1 y 10.

```
if nota >= 1 and nota <= 10:
    print("Nota válida.")
```

Otro ejemplo

```
edad = 25
```

```
if edad >= 18 and edad <= 65:  
    print("Edad permitida.")
```

Forma abreviada en Python

Python permite escribir la misma condición de manera más elegante.

En lugar de

```
if edad >= 18 and edad <= 65:
```

podemos escribir

```
if 18 <= edad <= 65:
```

Ambas formas son correctas.

Sin embargo, para estudiantes que recién comienzan, suele ser más fácil comprender primero la versión completa.

6. Combinando `and` y `or`

Supongamos el siguiente problema.

Un estudiante aprueba si:

- obtuvo al menos 7,

o bien

- obtuvo 6 y realizó el trabajo práctico.

Podemos escribir

```
if nota >= 7 or (nota == 6 and hizo_tp):  
    print("Aprobado.")
```

Observemos los paréntesis.

Sin ellos la condición resulta mucho más difícil de interpretar.

7. Prioridad de evaluación

Python sigue un orden específico para evaluar expresiones lógicas.

1. Paréntesis
2. `not`
3. `and`
4. `or`

Ejemplo

```
True or False and False
```

Python primero evalúa

```
False and False
```

Resultado

```
False
```

Luego

```
True or False
```

Resultado

```
True
```

Ejercicios

Ejercicio 1

Escribir una condición que determine si una persona tiene entre 18 y 30 años.

Ejercicio 2

Solicitar una nota y una asistencia. Mostrar "Aprobado" únicamente si la nota es mayor o igual a 7 y la asistencia es al menos del 75%.

Ejercicio 3

Solicitar un usuario y una contraseña. Permitir el acceso únicamente si el usuario es "admin" y la contraseña es "Python2026".

Ejercicio 4

Una biblioteca permite retirar libros si el usuario:

- es socio
- es docente.

Escribir el programa correspondiente.

Ejercicio 5

Una empresa concede un descuento si se cumple alguna de las siguientes condiciones:

- el cliente es socio y la compra supera los \$20.000;
- el cliente posee un cupón de descuento.

Solicitar los datos necesarios y determinar si corresponde aplicar el descuento.

B2.2.3 – Estructuras de repetición: `while`

1. ¿Qué es una estructura de repetición?

Hasta ahora nuestros programas podían:

- ejecutar instrucciones en secuencia;
- tomar decisiones mediante `if`.

Pero imaginemos el siguiente problema.

Queremos mostrar diez veces el mensaje:

```
Bienvenido al curso de Python
```

Una solución sería:

```
print("Bienvenido al curso de Python")
print("Bienvenido al curso de Python")
print("Bienvenido al curso de Python")
print("Bienvenido al curso de Python")
print("Bienvenido al curso de Python")
print("Bienvenido al curso de Python")
print("Bienvenido al curso de Python")
print("Bienvenido al curso de Python")
print("Bienvenido al curso de Python")
print("Bienvenido al curso de Python")
```

Aunque funciona, claramente no es una buena solución.

Estamos escribiendo el mismo código una y otra vez.

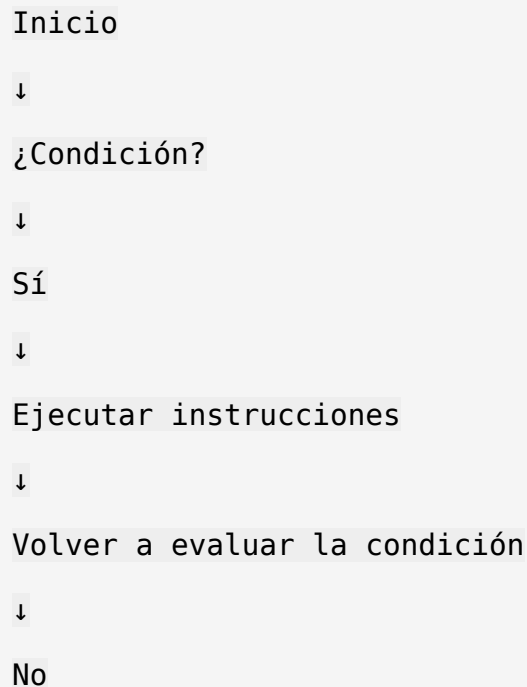
Para evitar esto aparecen las **estructuras repetitivas**, también llamadas **bucles** (*loops*).

¿Qué es un bucle?

Un **bucle** es una estructura de control que permite ejecutar un bloque de instrucciones varias veces.

En lugar de escribir diez veces la misma instrucción, el programa la ejecuta automáticamente.

Visualmente:





Continuar

¿Cuándo utilizar un bucle?

Siempre que una acción deba repetirse.

Por ejemplo:

- leer varios datos;
- recorrer una lista;
- calcular una suma;
- validar una contraseña;
- repetir un menú hasta que el usuario decida salir.

2. El bucle `while`

La palabra inglesa **while** significa

"mientras".

La idea es muy simple.

Mientras una condición sea verdadera, el bloque de instrucciones continuará ejecutándose.

Su sintaxis es:

```
while condición:  
    instrucciones
```

Observemos nuevamente dos detalles importantes.

- termina con dos puntos (:);
- el bloque debe estar correctamente indentado.

¿Cómo funciona?

Supongamos

```
contador = 1
while contador <= 3:
    print(contador)
    contador = contador + 1
```

Visualmente ocurre lo siguiente.

```
contador = 1
↓
¿1 <= 3?
↓
Sí
↓
Imprimir 1
↓
contador = 2
↓
¿2 <= 3?
↓
Sí
↓
Imprimir 2
↓
contador = 3
↓
¿3 <= 3?
↓
Sí
```

↓
Imprimir 3
↓
contador=4
↓
¿4<=3?
↓
No
↓
Fin

Salida

1
2
3

La condición se evalúa antes de cada repetición

Esto es muy importante.

El programa **no sabe** cuántas veces repetirá el bucle.

Simplemente pregunta una y otra vez:

¿La condición sigue siendo verdadera?

Si la respuesta es sí,

repite.

Si la respuesta es no,

termina.

3. El contador

En la mayoría de los bucles aparece una variable denominada **contador**.

Su función consiste en llevar la cuenta de las repeticiones.

Ejemplo

```
contador = 1
while contador <= 5:
    print("Hola")
    contador += 1
```

Resultado

```
Hola
Hola
Hola
Hola
Hola
```

¿Qué significa +=?

La instrucción

```
contador += 1
```

equivale exactamente a

```
contador = contador + 1
```

De la misma forma

```
total += numero
```

equivale a

```
total = total + numero
```

Es simplemente una forma abreviada muy utilizada en Python.

4. Tabla de seguimiento

Programa

```
contador = 1
while contador <= 4:
    print(contador)
    contador += 1
```

Iteración	contador antes	Se imprime	contador después
Inicio	1	-	1
1	1	1	2
2	2	2	3
3	3	3	4
4	4	4	5
Fin	5	-	-

5. Menús

Otra aplicación muy común.

```
opcion = ""
while opcion != "4":
    print("1. Alta")
    print("2. Baja")
    print("3. Consulta")
    print("4. Salir")

    opcion = input("Seleccione una opción: ")
print("Programa finalizado.")
```

Este tipo de estructura aparece constantemente en programas reales.

Ejercicios

Ejercicio 1

Mostrar los números del 1 al 20 utilizando un bucle `while`.

Ejercicio 2

Mostrar todos los números pares entre 2 y 50.

Ejercicio 3

Solicitar una contraseña y repetir la solicitud hasta que el usuario escriba `"PythonIB"`.

Ejercicio 4

Solicitar números enteros positivos. El programa finalizará cuando el usuario ingrese un número negativo. Al terminar, mostrar cuántos números positivos se ingresaron.

Ejercicio 5

Solicitar una nota hasta que el usuario ingrese un valor entre 1 y 10 inclusive.

B2.2.4 – Estructuras de repetición: `for`

1. ¿Por qué existe el `for`?

En el capítulo anterior utilizamos `while` para repetir instrucciones.

Por ejemplo:

```
contador = 1
while contador <= 5:
    print(contador)
```



```
contador += 1
```

Observemos que tuvimos que preocuparnos de tres cosas:

- crear un contador;
- escribir la condición;
- incrementar el contador.

En problemas sencillos esto funciona bien.

Sin embargo, cuando **sabemos exactamente cuántas veces queremos repetir una acción**, Python ofrece una estructura mucho más simple.

Esa estructura es el **bucle for**.

¿Cuándo utilizar `for`?

El `for` se utiliza cuando conocemos de antemano la cantidad de repeticiones.

Por ejemplo:

- Mostrar los números del 1 al 100.
- Leer las notas de 30 alumnos.
- Recorrer una lista.
- Dibujar una figura 20 veces.
- Repetir un experimento exactamente 10 veces.

En todos estos casos conocemos la cantidad de iteraciones antes de comenzar.

2. Sintaxis del `for`

La forma más común es

```
for variable in range(...):  
    instrucciones
```

La variable recibe automáticamente un nuevo valor en cada repetición.

Primer ejemplo

```
for i in range(5):  
    print(i)
```

Salida

```
0  
1  
2  
3  
4
```

¿Qué ocurrió?

La función

```
range(5)
```

genera la siguiente secuencia.

```
0  
↓  
1  
↓  
2  
↓  
3  
↓  
4
```

El número 5 **no se incluye**.

Esta es una característica muy importante de `range()`.

Flujo de ejecución

`i = 0`

↓

Ejecutar

↓

`i = 1`

↓

Ejecutar

↓

`i = 2`

↓

Ejecutar

↓

`i = 3`

↓

Ejecutar

↓

`i = 4`

↓

Ejecutar

↓

Fin

3. La función `range()`

`range()` puede utilizarse de tres maneras diferentes.

Primera forma

```
range(fin)
```

Comienza en cero.

Termina antes de llegar al valor indicado.

Ejemplo

```
range(5)
```

Produce

```
0 1 2 3 4
```

Segunda forma

```
range(inicio, fin)
```

Ejemplo

```
for i in range(3,8):  
    print(i)
```

Salida

```
3  
4  
5  
6  
7
```

Tercera forma

```
range(inicio, fin, paso)
```

Ejemplo

```
for i in range(2,11,2):  
    print(i)
```

Salida

```
2  
4  
6  
8  
10
```

El tercer parámetro indica cuánto aumenta el contador en cada iteración.

Paso negativo

También podemos contar hacia atrás.

```
for i in range(10,0,-1):  
    print(i)
```

Salida

```
10  
9  
8  
7  
6  
5  
4  
3  
2  
1
```

4. Comparación entre `while` y `for`

Con `while`

```
contador = 1  
while contador <= 5:  
    print(contador)  
    contador += 1
```

Con `for`

```
for contador in range(1,6):  
    print(contador)
```

Ambos programas producen exactamente el mismo resultado.

La diferencia es que el `for` resulta más corto y menos propenso a errores.

¿Cuál conviene utilizar?

Utilizar `while`

Cuando **no sabemos cuántas veces** se repetirá el proceso.

Ejemplos:

- validar una contraseña;
 - esperar que el usuario escriba un dato correcto;
 - mostrar un menú hasta que el usuario elija salir.
-

Utilizar `for`

Cuando **sí conocemos** la cantidad de repeticiones.

Ejemplos:

- imprimir los primeros 100 números;
 - recorrer una lista;
 - leer las notas de 30 estudiantes.
-

5. La variable de control

En

```
for i in range(5):
```

la variable

```
i
```

recibe automáticamente

0

↓

1

↓

2

↓

3

↓

4

No necesitamos modificarla manualmente.

Error frecuente

Muchos alumnos escriben

```
for i in range(5):  
    print(i)  
    i += 1
```

Esto es innecesario.

El propio `for` modifica la variable automáticamente.

6. Ejemplo: tabla de multiplicar

```
numero = int(input("Ingrese un número: "))  
for i in range(1,11):  
    print(numero,"x",i,"=",numero*i)
```

Si el usuario escribe

5

Obtendrá

5 x 1 = 5

5 x 2 = 10

...

5 x 10 = 50

7. Acumuladores con `for`

Supongamos que queremos sumar los primeros cinco números.

```
suma = 0
for i in range(1,6):
    suma += i
print(suma)
```

Tabla de seguimiento

Iteración	i	suma
Inicio	-	0
1	1	1
2	2	3
3	3	6
4	4	10
5	5	15

Resultado

15

8. Contadores

También podemos utilizar `for` para contar elementos.

Ejemplo

```
contador = 0
for i in range(10):
    contador += 1
print(contador)
```

Resultado

```
10
```

9. Bucles anidados

Un bucle puede contener otro bucle.

Ejemplo

```
for fila in range(3):
    for columna in range(4):
        print("*", end=" ")
    print()
```

Salida

```
* * * *
* * * *
* * * *
```

¿Cómo funciona?

El bucle exterior controla las filas.

El interior controla las columnas.

Visualmente

Fila 1

Columna 1

Columna 2

Columna 3

Columna 4

↓

Fila 2

...

↓

Fila 3

Ejercicios

Ejercicio 1

Mostrar los números del 1 al 100 utilizando un bucle `for`.

Ejercicio 2

Mostrar los números pares entre 2 y 50.

Ejercicio 3

Solicitar un número entero y mostrar su tabla de multiplicar del 1 al 10.

Ejercicio 4

Calcular la suma de los primeros 100 números naturales utilizando un bucle `for`.

Ejercicio 5

Solicitar cinco notas, calcular el promedio e indicar si el estudiante aprobó (promedio mayor o igual a 7).

Ejercicio 6

Mostrar la siguiente figura utilizando bucles anidados.

```
*  
**  
***  
****  
*****
```

Ejercicio 7

Solicitar un número entero positivo `n` y generar el siguiente patrón:

```
1  
1 2  
1 2 3  
1 2 3 4  
...  
1 2 ... n
```

B2.2.5 – Listas: creación y acceso a elementos

1. ¿Por qué necesitamos listas?

Hasta ahora utilizamos variables individuales.

Por ejemplo:

```
nota1 = 8  
nota2 = 7  
nota3 = 10
```

```
nota4 = 6
nota5 = 9
```

Esto funciona.

Pero imaginemos una clase con **35 estudiantes**.

Necesitaríamos escribir:

```
nota1
nota2
nota3
...
nota35
```

¿Y si fueran 500 alumnos?

Claramente esta solución no es práctica.

Necesitamos una estructura capaz de almacenar muchos valores utilizando **un solo nombre**.

Para eso existen las **listas**.

¿Qué es una lista?

Una **lista** es una estructura de datos que almacena una colección ordenada de elementos.

Todos los elementos pertenecen a la misma lista.

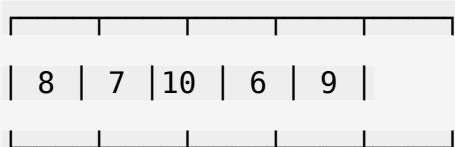
Ejemplo

```
notas = [8,7,10,6,9]
```

Ahora todas las notas están almacenadas en una única variable.

Visualmente

notas



Ventajas de utilizar listas

Las listas permiten:

- almacenar muchos datos relacionados;
- acceder fácilmente a cualquier elemento;
- recorrer todos los elementos mediante un bucle;
- agregar o eliminar información dinámicamente.

Por estas razones constituyen una de las estructuras más utilizadas en programación.

2. Crear listas

La forma más sencilla consiste en escribir los elementos entre corchetes.

```
numeros = [4,8,2,15,6]
```

Lista de nombres

```
nombres = ["Ana", "Luis", "Pedro", "María"]
```

Lista de números decimales

```
precios = [12.5, 18.3, 9.99]
```

Lista de valores booleanos

```
aprobados = [True, False, True]
```

Python también permite listas con distintos tipos de datos.

```
datos = ["Juan", 18, 7.5, True]
```

Aunque esto es posible, en la práctica suele ser recomendable que todos los elementos representen el mismo tipo de información para facilitar el procesamiento.

3. Los índices

Cada elemento posee una posición denominada **índice**.

Muy importante:

Los índices en Python comienzan en 0.

Ejemplo

```
colores = ["Rojo", "Azul", "Verde", "Amarillo"]
```

Visualmente

Índice

0	1	2	3
---	---	---	---

Rojo	Azul	Verde	Amarillo

El primer elemento tiene índice **0**, no 1.

Este es uno de los errores más frecuentes entre quienes comienzan a programar.

Acceder a un elemento

Para acceder a un elemento utilizamos el índice entre corchetes.

```
print(colores[0])
```

Salida

```
Rojo
```

```
print(colores[2])
```

Salida

```
Verde
```

¿Qué ocurre si el índice no existe?

Supongamos

```
colores = ["Rojo", "Azul", "Verde"]
```

Intentamos acceder a

```
print(colores[8])
```

Python responde

```
IndexError: list index out of range
```

El índice solicitado está fuera de los límites de la lista.

4. Modificar elementos

Las listas son **mutables**.

Esto significa que podemos cambiar sus elementos.

Ejemplo

```
notas = [6, 7, 8]  
notas[0] = 10  
print(notas)
```

Salida

```
[10, 7, 8]
```

Solo cambió el primer elemento.

Otro ejemplo

```
nombres = ["Ana", "Luis", "Pedro"]  
nombres[1] = "María"  
print(nombres)
```

Salida

```
['Ana', 'María', 'Pedro']
```

5. Obtener la cantidad de elementos

La función `len()` devuelve la cantidad de elementos de una lista.

```
frutas = ["Manzana", "Pera", "Banana", "Naranja"]  
print(len(frutas))
```

Salida

```
4
```

También puede utilizarse en condiciones.

```
if len(frutas) > 3:  
    print("La lista tiene más de tres elementos.")
```

6. Índices negativos

Python permite acceder a los elementos comenzando desde el final.

Ejemplo

```
días = ["Lunes", "Martes", "Miércoles", "Jueves"]
```


Índices positivos

0 1 2 3

Índices negativos

-4 -3 -2 -1

Ejemplo

```
print(dias[-1])
```

Salida

Jueves

```
print(dias[-2])
```

Salida

Miércoles

Los índices negativos son especialmente útiles cuando necesitamos acceder al último elemento sin conocer el tamaño de la lista.

Ejercicios

Ejercicio 1

Crear una lista con los nombres de cinco ciudades e imprimir el primer y el último elemento.

Ejercicio 2

Crear una lista con cinco números enteros e imprimir el tercer elemento.

Ejercicio 3

Crear una lista con cuatro colores. Cambiar el segundo color por otro elegido por el usuario y mostrar la lista actualizada.

Ejercicio 4

Crear una lista con las notas de cinco alumnos. Mostrar:

- la primera nota;
 - la última nota;
 - la cantidad total de notas almacenadas.
-

Ejercicio 5

Dada la lista:

```
numeros = [12, 8, 15, 20, 5]
```

Indicar:

- ¿Cuál es el valor de `numeros[0]`?
 - ¿Cuál es el valor de `numeros[3]`?
 - ¿Cuál es el valor de `numeros[-1]`?
 - ¿Cuántos elementos tiene la lista?
-

Ejercicio 6

Una tienda almacena los precios de cinco productos en una lista.

Desarrollar un programa que:

1. Cree la lista con los precios.
 2. Muestre el primer y el último precio.
 3. Modifique el precio del tercer producto.
 4. Muestre nuevamente la lista completa.
-

B2.2.6 – Recorrido de listas con `for` y `while`

1. ¿Qué significa recorrer una lista?

En el capítulo anterior aprendimos a acceder a un elemento específico.

Por ejemplo:

```
notas = [8, 6, 9, 10, 7]
print(notas[2])
```

Salida

9

Aquí estamos accediendo solamente al tercer elemento.

Pero imaginemos que queremos mostrar **todas** las notas.

Una solución sería

```
print(notas[0])
print(notas[1])
print(notas[2])
print(notas[3])
print(notas[4])
```

Funciona.

Pero...

¿Qué ocurre si la lista tiene 100 elementos?

¿Y si tiene 10.000?

Claramente necesitamos otra estrategia.

¿Qué es recorrer una lista?

Recorrer una lista significa

visitar uno por uno todos sus elementos.

Visualmente

Índice

0	1	2	3	4
---	---	---	---	---



↑

El programa pasa por cada posición.

2. Recorrido mediante `for`

La forma más utilizada en Python es

```
for elemento in lista:  
    instrucciones
```

Observemos el siguiente ejemplo.

```
notas = [8,6,9,10,7]  
  
for nota in notas:  
    print(nota)
```

Salida

```
8  
6  
9  
10  
7
```

¿Cómo funciona?

Primera iteración

```
nota = 8
```

Segunda

```
nota = 6
```

Tercera

```
nota = 9
```

Y así sucesivamente.

Cuando llega al último elemento,
el bucle termina automáticamente.

Ventajas

Esta forma es:

- sencilla;
- clara;
- muy utilizada en Python;
- evita errores con los índices.

Por eso suele ser la primera opción cuando solo necesitamos leer los elementos.

3. Recorrer utilizando índices

En muchas ocasiones necesitamos conocer la posición del elemento.

Para ello utilizamos

```
range(len(lista))
```

Ejemplo

```
notas = [8,6,9,10,7]
for i in range(len(notas)):
    print(i, notas[i])
```

Salida

```
0 8
1 6
2 9
3 10
4 7
```

¿Qué significa `len(notas)`?

Recordemos

```
len(notas)
```

devuelve

```
5
```

Por lo tanto

```
range(len(notas))
```

equivale a

```
range(5)
```

que genera

```
0
1
2
3
4
```

Es decir,

todos los índices válidos de la lista.

Diagrama del recorrido

```
i = 0
↓
notas[0]
↓
i = 1
↓
notas[1]
↓
i = 2
↓
notas[2]
↓
...
↓
Fin
```

4. Recorrido mediante `while`

También podemos recorrer una lista utilizando `while`.

Ejemplo

```
notas = [8,6,9,10,7]
i = 0
while i < len(notas):
    print(notas[i])
    i += 1
```

Obtendremos exactamente la misma salida.

¿Cómo funciona?

`i =0`

↓

`¿0<5?`

↓

Sí

↓

Mostrar `notas[0]`

↓

`i=1`

↓

`¿1<5?`

↓

Sí

↓

Mostrar `notas[1]`

↓

...

↓

`i=5`

↓

`¿5<5?`

↓

No

↓

Fin

Comparación

Con `for`

```
for nota in notas:  
    print(nota)
```

Con `while`

```
i = 0  
while i < len(notas):  
    print(notas[i])  
    i += 1
```

Ambos programas hacen exactamente lo mismo.

Sin embargo,

el primero resulta mucho más sencillo.

¿Cuál conviene utilizar?

Utilizar `for`

Cuando simplemente queremos recorrer todos los elementos.

Es la opción recomendada.

Utilizar `while`

Cuando necesitamos controlar manualmente el índice o el recorrido.

Por ejemplo

- avanzar de dos en dos;
 - detener el recorrido antes del final;
 - recorrer en sentido inverso mediante un algoritmo propio.
-

5. Modificar elementos durante el recorrido

También podemos cambiar los valores.

Ejemplo

```
notas = [5,6,7,8]
for i in range(len(notas)):
    notas[i] += 1
print(notas)
```

Resultado

```
[6,7,8,9]
```

Cada elemento aumentó una unidad.

Otro ejemplo

Convertir todos los nombres a mayúsculas.

```
nombres = ["Ana", "Luis", "Pedro"]
for i in range(len(nombres)):
    nombres[i] = nombres[i].upper()
print(nombres)
```

Resultado

```
['ANA', 'LUIS', 'PEDRO']
```

6. Recorrer en sentido inverso

Podemos recorrer desde el último elemento hasta el primero.

```
numeros = [10,20,30,40]
for i in range(len(numeros)-1,-1,-1):
    print(numeros[i])
```

Salida

```
40
30
20
10
```

Explicación

Supongamos

```
len(numeros)=4
```

Entonces

```
range(3,-1,-1)
```

produce

```
3
2
1
0
```

Justamente los índices de la lista en orden inverso.

Ejercicios

Ejercicio 1

Crear una lista con cinco nombres e imprimir cada uno utilizando un bucle `for`.

Ejercicio 2

Crear una lista con diez números e imprimir únicamente aquellos mayores que 50.

Ejercicio 3

Crear una lista con cinco números. Multiplicar cada elemento por 2 y mostrar la lista modificada.

Ejercicio 4

Crear una lista con los meses del año e imprimirlos en orden inverso.

Ejercicio 5

Solicitar cinco notas al usuario, almacenarlas en una lista y luego mostrarlas una por una utilizando un `for`.

Ejercicio 6

Recorrer una lista de edades utilizando un `while` e imprimir únicamente las edades mayores o iguales a 18.

Ejercicio 7

Una tienda almacena las ventas de una semana en una lista de siete elementos.

Desarrollar un programa que:

1. Solicite las siete ventas y las almacene en una lista.
2. Muestre todas las ventas.
3. Muestre el número de día (índice) junto con el importe correspondiente.
4. Muestre las ventas en orden inverso.

B2.2.7 – Métodos de listas

1. ¿Qué es un método?

Hasta ahora utilizamos funciones como

```
len(lista)
```

Las funciones reciben información y devuelven un resultado.

Pero las listas también poseen **métodos**.

Un método es una función que pertenece a un objeto.

Su sintaxis es

```
lista.metodo()
```

Por ejemplo

```
nombres.append("Ana")
```

Aquí el método `append()` pertenece a la lista `nombres`.

Función vs Método

Función	Método
<code>len(lista)</code>	<code>lista.append(5)</code>
Se escribe fuera del objeto	Se escribe después del objeto
Devuelve un resultado	Generalmente modifica el objeto

2. Agregar elementos: `append()`

Es el método más utilizado.

Permite agregar un elemento **al final** de la lista.

Ejemplo

```
frutas = ["Manzana", "Pera"]  
frutas.append("Banana")  
print(frutas)
```

Resultado

```
['Manzana', 'Pera', 'Banana']
```

Visualmente

Antes

```
["Manzana", "Pera"]
```

↓

Después de `append("Banana")`

```
["Manzana", "Pera", "Banana"]
```

Otro ejemplo

```
numeros = [10, 20]  
numeros.append(30)  
numeros.append(40)  
print(numeros)
```

Resultado

```
[10, 20, 30, 40]
```

3. Insertar elementos: `insert()`

Si queremos agregar un elemento en una posición específica utilizamos

```
lista.insert(indice, elemento)
```

Ejemplo

```
colores = ["Rojo","Verde"]
colores.insert(1,"Azul")
print(colores)
```

Resultado

```
['Rojo', 'Azul', 'Verde']
```

Observemos que los demás elementos se desplazan una posición hacia la derecha.

Visualmente

Antes

0	1
Rojo	Verde

Después

0	1	2
Rojo	Azul	Verde

4. Eliminar por valor: `remove()`

El método `remove()` elimina la primera aparición del valor indicado.

Ejemplo

```
animales = ["Perro","Gato","Conejo"]
animales.remove("Gato")
print(animales)
```

Resultado

```
['Perro', 'Conejo']
```

¿Qué ocurre si el elemento no existe?

```
animales.remove("León")
```

Produce

```
ValueError
```

Porque ese elemento no está en la lista.

5. Eliminar por posición: `pop()`

`pop()` elimina un elemento utilizando su índice.

Ejemplo

```
numeros = [10,20,30,40]
numeros.pop(2)
print(numeros)
```

Resultado

```
[10,20,40]
```

El elemento de índice 2 (30) desapareció.

`pop()` también devuelve el elemento eliminado

```
numeros = [10,20,30]
eliminado = numeros.pop(1)
print(eliminado)
print(numeros)
```

Salida


```
20
```

```
[10,30]
```

`pop()` sin parámetros

Si no se indica un índice, elimina el último elemento.

```
lista = [1,2,3]
lista.pop()
print(lista)
```

Resultado

```
[1,2]
```

6. Buscar un elemento: `index()`

Devuelve la posición de un elemento.

Ejemplo

```
dias = ["Lunes","Martes","Miércoles"]
print(dias.index("Martes"))
```

Resultado

```
1
```

Si el elemento no existe

```
dias.index("Domingo")
```

Produce

```
ValueError
```

7. Saber si un elemento existe: `in`

Muchas veces es más conveniente preguntar si un elemento pertenece a la lista.

```
frutas = ["Manzana", "Banana", "Pera"]  
print("Banana" in frutas)
```

Resultado

```
True
```

```
print("Kiwi" in frutas)
```

Resultado

```
False
```

Este operador evita errores cuando luego queremos utilizar `remove()` o `index()`.

8. Contar apariciones: `count()`

Permite saber cuántas veces aparece un elemento.

```
numeros = [2,5,2,8,2]  
print(numeros.count(2))
```

Resultado

```
3
```

9. Ordenar listas: `sort()`

Ordena la lista de menor a mayor.

```
numeros = [8,2,5,1]
numeros.sort()
print(numeros)
```

Resultado

```
[1,2,5,8]
```

También funciona con texto.

```
nombres = ["Luis", "Ana", "Pedro"]
nombres.sort()
print(nombres)
```

Resultado

```
['Ana', 'Luis', 'Pedro']
```

Orden descendente

```
numeros.sort(reverse=True)
print(numeros)
```

Resultado

```
[8,5,2,1]
```

10. Invertir el orden: `reverse()`

No ordena.

Simplemente cambia el orden actual.

```
numeros = [10,20,30,40]
numeros.reverse()
```

```
print(numeros)
```

Resultado

```
[40,30,20,10]
```

Observemos la diferencia.

Lista original

```
8 2 5 1
```

Después de `sort()`

```
1 2 5 8
```

Después de `reverse()`

```
1 5 2 8
```

(si previamente estaba ordenada).

`reverse()` **no ordena**.

Solo invierte.

11. Vaciar una lista: `clear()`

```
datos = [5,7,9]
```

```
datos.clear()
```

```
print(datos)
```

Resultado

```
[]
```

12. Copiar listas

Muchos alumnos escriben

```
lista2 = lista1
```

Pensando que obtienen una copia.

En realidad ambas variables apuntan a la misma lista.

```
lista1 = [1,2,3]
lista2 = lista1
lista2.append(4)
print(lista1)
```

Resultado

```
[1,2,3,4]
```

La forma correcta es

```
lista2 = lista1.copy()
```

Ahora sí son listas independientes.

Resumen de métodos

Método	Función
<code>append()</code>	Agrega al final
<code>insert()</code>	Inserta en una posición
<code>remove()</code>	Elimina por valor
<code>pop()</code>	Elimina por índice
<code>index()</code>	Devuelve la posición
<code>count()</code>	Cuenta apariciones
<code>sort()</code>	Ordena la lista
<code>reverse()</code>	Invierte el orden
<code>clear()</code>	Vacía la lista
<code>copy()</code>	Crea una copia

Ejercicios

Ejercicio 1

Crear una lista vacía y agregar cinco números utilizando `append()`.

Ejercicio 2

Crear una lista con tres nombres e insertar otro nombre en la segunda posición.

Ejercicio 3

Crear una lista con cinco colores. Solicitar un color al usuario y eliminarlo si existe en la lista. Si no existe, mostrar un mensaje indicando que no se encontró.

Ejercicio 4

Crear una lista de diez números. Ordenarla de menor a mayor y luego de mayor a menor.

Ejercicio 5

Crear una lista con diez números enteros y mostrar cuántas veces aparece un número ingresado por el usuario.

Ejercicio 6

Crear una lista con los nombres de cinco estudiantes. Realizar una copia utilizando `copy()`, agregar un nuevo estudiante únicamente a la copia y mostrar ambas listas para comprobar que son independientes.

Ejercicio 7

Desarrollar un programa que mantenga una lista de productos de una tienda. El programa debe mostrar un menú con las siguientes opciones:

1. Agregar un producto.
2. Eliminar un producto.
3. Mostrar todos los productos.
4. Ordenar alfabéticamente la lista.
5. Buscar si un producto existe.
6. Salir.

El menú deberá repetirse hasta que el usuario seleccione la opción **6**.

B2.2.8 – Algoritmos sobre listas

1. ¿Qué es un algoritmo sobre una lista?

Hasta ahora aprendimos a:

- crear listas;
- recorrerlas;
- agregar o eliminar elementos.

Pero normalmente **no almacenamos datos solamente para guardarlos**.

Queremos obtener información.

Por ejemplo:

- ¿Cuál es la nota más alta?
- ¿Cuál es el promedio?
- ¿Cuántos alumnos aprobaron?
- ¿Existe un alumno llamado "Juan"?
- ¿Cuál fue la temperatura máxima?

Todas estas preguntas se responden recorriendo la lista mediante un algoritmo.

2. Calcular la suma

Supongamos la siguiente lista.

```
notas = [8, 6, 10, 7, 9]
```

Queremos calcular la suma de todas las notas.

Primero necesitamos una variable acumuladora.

```
suma = 0
```

Luego recorremos la lista.

```
notas = [8,6,10,7,9]
```

```
suma = 0
```

```
for nota in notas:
```

```
    suma += nota
```

```
print(suma)
```

Resultado

```
40
```

¿Qué hace el acumulador?

Veámoslo paso a paso.

Iteración	Nota	Suma
Inicio	-	0
1	8	8
2	6	14
3	10	24
4	7	31
5	9	40

El acumulador conserva el resultado parcial en cada iteración.

3. Calcular el promedio

Una vez obtenida la suma,
el promedio es muy sencillo.

```
notas = [8,6,10,7,9]
suma = 0
for nota in notas:
    suma += nota
promedio = suma / len(notas)
print(promedio)
```

Resultado

8.0

¿Por qué utilizamos `len()`?

Porque el promedio se calcula mediante

Promedio = Suma / Cantidad

La cantidad de elementos la proporciona

`len(notas)`

4. Encontrar el mayor

Uno de los algoritmos más importantes.

Supongamos

```
numeros = [15,8,27,13,20]
```

¿Cómo encontramos el mayor?

La idea consiste en suponer inicialmente que el primero es el mayor.

```
mayor = numeros[0]
```

Luego comparamos todos los demás.

```
numeros = [15, 8, 27, 13, 20]
mayor = numeros[0]
for numero in numeros:
    if numero > mayor:
        mayor = numero
print(mayor)
```

Resultado

```
27
```

Seguimiento

Número	Mayor
15	15
8	15
27	27
13	27
20	27

¿Por qué no inicializar con cero?

Muchos alumnos escriben

```
mayor = 0
```

Esto falla cuando todos los números son negativos.

Ejemplo

```
[-8, -4, -12]
```

El algoritmo devolvería incorrectamente

0

Por eso siempre inicializamos con el primer elemento de la lista.

5. Encontrar el menor

Es exactamente el mismo algoritmo.

```
numeros = [15,8,27,13,20]
menor = numeros[0]
for numero in numeros:
    if numero < menor:
        menor = numero
print(menor)
```

Resultado

8

6. Contar elementos

Supongamos

```
notas = [5,8,9,6,10,7]
```

Queremos saber cuántos aprobaron.

```
contador = 0
for nota in notas:
    if nota >= 7:
        contador += 1
```

```
print(contador)
```

Resultado

4

Seguimiento

Nota	¿Aprueba?	Contador
5	No	0
8	Sí	1
9	Sí	2
6	No	2
10	Sí	3
7	Sí	4

7. Buscar un elemento (Búsqueda lineal)

Este algoritmo aparece frecuentemente en el IB.

Supongamos

```
nombres = ["Ana", "Luis", "Pedro", "María"]
```

Queremos saber si existe

Pedro

Podemos hacer

```
encontrado = False
for nombre in nombres:
    if nombre == "Pedro":
        encontrado = True
if encontrado:
    print("Existe.")
```

```
else:  
    print("No existe.")
```

Resultado

Existe.

¿Por qué se llama búsqueda lineal?

Porque el algoritmo revisa los elementos uno por uno.

Ana
↓
Luis
↓
Pedro
↓
María

En el peor caso debe recorrer toda la lista.

8. Buscar la posición

No solo queremos saber si existe.

También queremos conocer su índice.

```
nombres = ["Ana", "Luis", "Pedro", "María"]  
indice = -1  
for i in range(len(nombres)):  
    if nombres[i] == "Pedro":  
        indice = i
```

```
break  
print(indice)
```

Resultado

2

¿Qué hace `break`?

La instrucción

```
break
```

interrumpe inmediatamente el bucle.

Visualmente

```
Ana  
↓  
Luis  
↓  
Pedro  
↓  
¡Encontrado!  
↓  
Salir del bucle
```

No continúa revisando los demás elementos.

9. Algoritmos combinados

Supongamos que queremos calcular

- promedio;
- máximo;
- mínimo.

Podemos hacerlo en un único recorrido.

```
notas = [8,6,10,7,9]
suma = 0
mayor = notas[0]
menor = notas[0]
for nota in notas:
    suma += nota
    if nota > mayor:
        mayor = nota
    if nota < menor:
        menor = nota
promedio = suma / len(notas)
print(promedio)
print(mayor)
print(menor)
```

¿Por qué es mejor?

Porque la lista se recorre una sola vez.

Esto hace el algoritmo más eficiente.

Ejercicios

Ejercicio 1

Solicitar 10 números enteros, almacenarlos en una lista y calcular su suma.

Ejercicio 2

Solicitar las notas de 8 estudiantes, almacenarlas en una lista y calcular el promedio del curso.

Ejercicio 3

Dada una lista de temperaturas, determinar la temperatura máxima y la mínima.

Ejercicio 4

Solicitar una lista de edades y mostrar cuántas personas son mayores o iguales a 18 años.

Ejercicio 5

Solicitar una lista de nombres y un nombre a buscar. Indicar si el nombre existe en la lista utilizando una búsqueda lineal.

Ejercicio 6

Solicitar una lista de números e indicar la posición del primer número par encontrado. Si no existe ninguno, mostrar un mensaje adecuado.

Ejercicio 7

Una tienda registra las ventas diarias de un mes (30 valores) en una lista. Desarrollar un programa que:

1. Solicite las 30 ventas y las almacene en una lista.
2. Calcule el total vendido durante el mes.
3. Calcule el promedio diario de ventas.
4. Determine el mayor y el menor importe vendido.
5. Cuente cuántos días las ventas superaron los \$100.000.

6. Muestre el día (índice) correspondiente a la venta más alta.

B2.2.9 – Listas bidimensionales (Matrices)

1. ¿Qué es una lista bidimensional?

Hasta ahora trabajamos con listas como:

```
notas = [8, 6, 9, 10]
```

Esto es una **lista unidimensional** (una sola fila de datos).

Pero muchas veces los datos se organizan en forma de tabla:

Alumno	Parcial 1	Parcial 2
Ana	8	9
Luis	6	7
Pedro	10	8

Para representar esto en programación necesitamos una estructura más poderosa:

Listas dentro de listas (matrices)

2. Creación de una matriz

Ejemplo:

```
notas = [  
    [8, 9],  
    [6, 7],  
    [10, 8]  
]
```

Visualmente:

	Parcial1	Parcial2
Ana	8	9
Luis	6	7
Pedro	10	8

3. Acceso a elementos

Para acceder usamos **dos índices**:

```
print(notas[0][0])
```

Salida:

```
8
```

¿Qué significa?

```
notas[fila][columna]
```

- Primer índice → fila
 - Segundo índice → columna
-

Ejemplos:

```
print(notas[1][0]) # Luis, Parcial 1  
print(notas[2][1]) # Pedro, Parcial 2
```

4. Modificar elementos

Las matrices también son mutables.

```
notas[0][1] = 10  
print(notas)
```

Resultado:

```
[[8, 10], [6, 7], [10, 8]]
```

5. Recorrer una matriz

Para recorrer una matriz necesitamos **dos bucles**.

Esto es fundamental en IB.

Ejemplo básico

```
for fila in notas:  
    for elemento in fila:  
        print(elemento)
```

Salida:

```
8  
9  
6  
7  
10  
8
```

Explicación

- El primer **for** recorre filas.
 - El segundo **for** recorre columnas dentro de cada fila.
-

6. Recorrer con índices (forma IB)

Cuando necesitamos posiciones:

```
for i in range(len(notas)):  
    for j in range(len(notas[i])):  
        print("Fila:", i, "Columna:", j, "Valor:", notas[i][j])
```

Salida:

```
Fila: 0 Columna: 0 Valor: 8  
Fila: 0 Columna: 1 Valor: 9  
Fila: 1 Columna: 0 Valor: 6  
Fila: 1 Columna: 1 Valor: 7
```

```
Fila: 2 Columna: 0 Valor: 10  
Fila: 2 Columna: 1 Valor: 8
```

7. Aplicación: suma de todos los elementos

```
suma = 0  
  
for fila in notas:  
    for elemento in fila:  
        suma += elemento  
  
print(suma)
```

8. Promedio general de una matriz

```
suma = 0  
cantidad = 0  
  
for fila in notas:  
    for elemento in fila:  
        suma += elemento  
        cantidad += 1  
  
promedio = suma / cantidad  
print(promedio)
```

9. Máximo en una matriz

```
mayor = notas[0][0]  
  
for fila in notas:  
    for elemento in fila:  
        if elemento > mayor:  
            mayor = elemento  
  
print(mayor)
```

10. Ejemplo IB: promedio por alumno

Cada fila representa un alumno.

```
for fila in notas:
    suma = 0

    for nota in fila:
        suma += nota

    promedio = suma / len(fila)

    print("Promedio:", promedio)
```

11. Matrices con `range()`

```
for i in range(len(notas)):
    for j in range(len(notas[i])):
        print(notas[i][j])
```

12. Representación visual de una matriz

```
i → filas
j → columnas
```

	j=0	j=1
i=0	8	9
i=1	6	7
i=2	10	8

Ejercicios

Ejercicio 1

Crear una matriz de 3x3 con números del 1 al 9 e imprimirla.

Ejercicio 2

Dada una matriz de notas, calcular la suma total de todos los elementos.

Ejercicio 3

Calcular el promedio de cada fila (cada alumno).

Ejercicio 4

Encontrar el número mayor en una matriz.

Ejercicio 5

Contar cuántos números son mayores a 7 en una matriz.

Ejercicio 6

Mostrar la matriz en formato de tabla (filas y columnas alineadas).

Ejercicio 7

Una escuela registra las notas de 4 estudiantes en 3 materias.

Desarrollar un programa que:

1. Almacene las notas en una matriz.
 2. Calcule el promedio de cada estudiante.
 3. Calcule el promedio de cada materia.
 4. Determine el mejor estudiante (mayor promedio).
-

B2.2.10 – Estructuras de datos: Pilas y Colas (LIFO y FIFO)

1. ¿Qué son las estructuras de datos?

Hasta ahora trabajamos con listas como estructuras generales.

Sin embargo, en programación no solo importa qué datos se almacenan, sino también cómo se organizan y cómo se accede a ellos.

A esto se lo denomina estructuras de datos.

Dos de las más importantes son:

- pilas (stack)
- colas (queue)

Ambas pueden implementarse utilizando listas en Python.

2. PILA (STACK) – LIFO

Definición

LIFO significa Last In, First Out.

Esto quiere decir que el último elemento en entrar es el primero en salir.

Ejemplo cotidiano

Una pila de platos:

- se coloca el plato 1
- luego el plato 2
- luego el plato 3

El primero en retirarse será el plato 3.

Representación

Entrada:

10 → 20 → 30

Salida:

30 → 20 → 10

Implementación en Python

```
pila = []
```

Agregar elementos (append)

```
pila.append(10)
pila.append(20)
pila.append(30)

print(pila)
```

Salida:

```
[10, 20, 30]
```

Eliminar elementos (pop)

```
pila.pop()

print(pila)
```

Salida:

```
[10, 20]
```

Funcionamiento LIFO

Se insertan:

10 → 20 → 30

Se eliminan:

30 → 20 → 10

Uso típico

- historial de navegación
 - deshacer acciones (Ctrl + Z)
 - ejecución de llamadas de funciones
-

3. COLA (QUEUE) – FIFO

Definición

FIFO significa First In, First Out.

Esto quiere decir que el primer elemento en entrar es el primero en salir.

Ejemplo cotidiano

Fila en un banco:

- llega Ana
- llega Luis
- llega Pedro

Ana es atendida primero.

Representación

Entrada:

Ana → Luis → Pedro

Salida:

Ana → Luis → Pedro

Implementación en Python

```
cola = []
```

Agregar elementos (append)

```
cola.append("Ana")  
cola.append("Luis")  
cola.append("Pedro")
```

```
print(cola)
```

Salida:

```
["Ana", "Luis", "Pedro"]
```

Eliminar elementos (pop)

```
cola.pop(0)  
  
print(cola)
```

Salida:

```
["Luis", "Pedro"]
```

Funcionamiento FIFO

Entran:

Ana → Luis → Pedro

Salen:

Ana → Luis → Pedro

4. Comparación entre pila y cola

Estructura	Principio	Inserción	Eliminación
Pila	LIFO	append()	pop()
Cola	FIFO	append()	pop(0)

5. Diferencia conceptual

- En la pila se accede por el final.
 - En la cola se accede por el inicio.
-

6. Ejemplo práctico

Pila

```
pila = []
```

```
pila.append("A")
pila.append("B")
pila.append("C")

print(pila.pop())
print(pila.pop())
print(pila.pop())
```

Salida:

```
C
B
A
```

Cola

```
cola = []

cola.append("A")
cola.append("B")
cola.append("C")

print(cola.pop(0))
print(cola.pop(0))
print(cola.pop(0))
```

Salida:

```
A
B
C
```

Ejercicios

Ejercicio 1

Simular una pila con cinco elementos y mostrar el orden de salida.

Ejercicio 2

Simular una cola de atención de clientes.

Ejercicio 3

Identificar si distintos casos corresponden a pila o cola.

Ejercicio 4

Indicar qué estructura corresponde a cada situación:

- historial de navegador
- fila de banco
- deshacer acciones
- impresión de documentos

Ejercicio 5

Implementar un sistema de turnos donde los clientes se atienden en orden de llegada.

CAPÍTULO 3: Modularización del código

B2.3.1 – Funciones en Python

1. ¿Qué problema resuelven las funciones?

Hasta ahora, todos los programas que escribimos tienen un problema:

- el código crece mucho
- se repite lógica
- es difícil de mantener

Ejemplo:

```
print("Bienvenido")
print("Ingrese un número")
print("Calculando...")
print("Resultado mostrado")
```

Si este bloque se repite varias veces, el programa se vuelve desordenado.

¿Qué es una función?

Una función es un bloque de código que:

- realiza una tarea específica
 - puede reutilizarse
 - puede recibir datos
 - puede devolver resultados
-

2. Definición de una función

En Python se define con `def`:

```
def saludar():  
    print("Hola estudiante")
```

Llamada a la función

```
saludar()  
saludar()  
saludar()
```

Salida:

```
Hola estudiante  
Hola estudiante  
Hola estudiante
```

3. Estructura general

```
def nombre_funcion():  
    instrucciones
```

y luego:

```
nombre_funcion()
```

4. Funciones con parámetros

Los parámetros permiten enviar información a la función.

Ejemplo:

```
def saludar(nombre):  
    print("Hola", nombre)
```

Llamada

```
saludar("Ana")  
saludar("Luis")
```

Salida:

```
Hola Ana  
Hola Luis
```

5. Más de un parámetro

```
def sumar(a, b):  
    print(a + b)
```

Llamada

```
sumar(5, 3)  
sumar(10, 7)
```

Salida:

```
8  
17
```

6. Funciones con retorno (`return`)

Hasta ahora las funciones solo mostraban resultados.

Pero muchas veces necesitamos **guardar el resultado**.

Ejemplo

```
def sumar(a, b):  
    return a + b
```

Uso

```
resultado = sumar(5, 3)  
print(resultado)
```

Salida:

8

¿Qué hace `return`?

- devuelve un valor
 - finaliza la función
 - permite reutilizar el resultado
-

7. Diferencia importante

Sin return

```
def sumar(a, b):  
    print(a + b)
```

→ solo muestra el resultado

Con return

```
def sumar(a, b):  
    return a + b
```

→ permite guardar el resultado

8. Ejemplo IB: promedio

```
def promedio(n1, n2, n3):  
    return (n1 + n2 + n3) / 3
```

Uso

```
print(promedio(8, 6, 10))
```

9. Variables locales

Las variables dentro de una función:

- solo existen dentro de la función
- no se pueden usar afuera

Ejemplo:

```
def ejemplo():  
    x = 10  
    print(x)
```

Esto es importante para evitar errores de alcance.

10. Flujo de ejecución

```
Programa principal  
↓  
Llamada a función  
↓  
Se ejecuta la función  
↓  
return (si existe)  
↓  
Vuelve al programa principal
```

11. Ejemplo completo

```
def calcular_descuento(precio, porcentaje):  
    return precio - (precio * porcentaje / 100)  
  
precio_final = calcular_descuento(1000, 10)  
print(precio_final)
```

Salida:

900.0

Ejercicios

Ejercicio 1

Crear una función que reciba un número e indique si es par o impar.

Ejercicio 2

Crear una función que calcule el área de un rectángulo.

Ejercicio 3

Crear una función que reciba tres números y devuelva el mayor.

Ejercicio 4

Crear una función que reciba una lista y devuelva su suma.

Ejercicio 5

Crear una función que calcule el factorial de un número.

Ejercicio 6

Diseñar un programa modular que permita:

- ingresar datos
- calcular promedio
- determinar si el alumno aprueba

utilizando funciones separadas para cada tarea.

B2.3.2 – Funciones con parámetros en Python

1. ¿Qué es un parámetro?

Hasta ahora las funciones que vimos no recibían información externa.

Ejemplo:

```
def saludar():  
    print("Hola")
```

Pero muchas veces necesitamos que una función trabaje con distintos datos.

Por ejemplo:

- saludar a distintos alumnos
- sumar distintos números
- calcular distintos promedios

Para eso usamos parámetros.

2. Definición de función con parámetros

Un parámetro es una variable que se define dentro de los paréntesis de la función.

```
def saludar(nombre):  
    print("Hola", nombre)
```

3. Llamada a la función con argumento

```
saludar("Ana")  
saludar("Luis")
```

Salida:

```
Hola Ana  
Hola Luis
```

4. Parámetro vs argumento

Concepto	Definición
Parámetro	variable en la definición de la función
Argumento	valor que se envía al llamar la función

Ejemplo:

```
def saludar(nombre):    # parámetro
    print(nombre)

saludar("Pedro")        # argumento
```

5. Funciones con múltiples parámetros

```
def sumar(a, b):
    print(a + b)
```

Llamada

```
sumar(5, 3)
sumar(10, 20)
```

Salida:

```
8
30
```

6. Ejemplo con más de dos parámetros

```
def promedio(n1, n2, n3):
    print((n1 + n2 + n3) / 3)
```

Llamada

```
promedio(8, 6, 10)
```

Salida:

8.0

7. Uso de parámetros en estructuras reales

Ejemplo: calcular descuento

```
def descuento(precio, porcentaje):  
    print(precio - (precio * porcentaje / 100))
```

Llamada

```
descuento(1000, 10)  
descuento(500, 20)
```

8. Parámetros y variables locales

Las variables creadas dentro de una función:

- solo existen dentro de la función
- no pueden usarse fuera

Ejemplo:

```
def ejemplo(x):  
    y = x + 10  
    print(y)
```

Esto significa que `y` no existe fuera de la función.

9. Flujo de ejecución con parámetros

```
Programa principal
↓
Llamada a función con datos
↓
Se asignan los parámetros
↓
Se ejecuta la función
↓
Retorno al programa principal
```

10. Ejemplo completo

```
def calcular_area(base, altura):
    print(base * altura)

calcular_area(5, 10)
calcular_area(3, 7)
```

Salida:

```
50
21
```

11. Ejercicios

Ejercicio 1

Crear una función que reciba un nombre y una edad, e imprima un mensaje personalizado.

Ejercicio 2

Crear una función que reciba dos números y muestre el mayor.

Ejercicio 3

Crear una función que calcule el perímetro de un rectángulo.

Ejercicio 4

Crear una función que reciba tres notas y devuelva el promedio.

Ejercicio 5

Diseñar un programa con funciones que:

- reciban datos de estudiantes
 - calculen su promedio
 - indiquen si aprueban o desaprueban
-

B2.3.3 – Funciones con retorno (`return`)

1. ¿Qué significa retornar un valor?

Hasta ahora nuestras funciones realizaban acciones como mostrar información en pantalla:

```
def sumar(a, b):  
    print(a + b)
```

Pero muchas veces necesitamos **guardar el resultado** para usarlo después.

Para eso utilizamos `return`.

2. Función con return

```
def sumar(a, b):  
    return a + b
```

Uso

```
resultado = sumar(5, 3)  
print(resultado)
```

Salida:

8

3. ¿Qué hace return?

El `return` cumple dos funciones:

- Devuelve un valor al programa principal
- Finaliza la ejecución de la función

Ejemplo:

```
def ejemplo():  
    return 10  
    print("Esto no se ejecuta")
```

4. Diferencia entre print y return

Con print

```
def sumar(a, b):  
    print(a + b)
```

- Solo muestra el resultado
 - No se puede reutilizar
-

Con return

```
def sumar(a, b):  
    return a + b
```

- Permite guardar el resultado
 - Permite reutilizarlo
-

5. Uso en variables

```
def multiplicar(a, b):  
    return a * b  
  
x = multiplicar(4, 5)  
y = multiplicar(2, 3)  
  
print(x)  
print(y)
```

Salida:

```
20  
6
```

6. Uso de return en cálculos complejos

Ejemplo: promedio

```
def promedio(a, b, c):  
    return (a + b + c) / 3
```

Uso

```
resultado = promedio(8, 6, 10)  
print(resultado)
```

7. Funciones dentro de funciones

```
def sumar(a, b):  
    return a + b  
  
def duplicar(x):  
    return x * 2  
  
resultado = duplicar(sumar(3, 4))  
print(resultado)
```

Salida:

14

8. Flujo de ejecución con return

```
Programa principal
↓
Llamada a función
↓
Cálculo interno
↓
return (envía resultado)
↓
Se guarda en variable
↓
Se continúa el programa
```

9. Ejemplo IB típico

```
def calcular_descuento(precio, porcentaje):
    return precio - (precio * porcentaje / 100)

precio_final = calcular_descuento(1000, 15)

print(precio_final)
```

10. Uso de return en decisiones

```
def es_mayor(a, b):
    if a > b:
        return a
    else:
        return b
```

Uso

```
print(es_mayor(10, 5))
```

11. Ejercicios

Ejercicio 1

Crear una función que reciba dos números y devuelva su suma.

Ejercicio 2

Crear una función que reciba un número y devuelva su cuadrado.

Ejercicio 3

Crear una función que reciba tres números y devuelva el mayor.

Ejercicio 4

Crear una función que calcule el área de un triángulo.

Ejercicio 5

Diseñar un programa con funciones que:

- reciban notas de estudiantes
 - calculen el promedio
 - devuelvan si el alumno aprueba o desapueba
-

B2.3.4 – Diseño de soluciones modulares

1. ¿Qué es un problema integrado?

Un problema integrado es aquel que no se resuelve con un solo concepto, sino combinando varios elementos de programación.

En el contexto del IB, estos problemas suelen incluir:

- listas o matrices
- funciones
- estructuras condicionales
- bucles
- cálculos y análisis de datos

El objetivo no es solo programar, sino **diseñar una solución completa y estructurada**.

2. Estrategia de resolución (modelo IB)

Para resolver este tipo de problemas se recomienda seguir siempre los siguientes pasos:

1. Análisis del problema

- ¿Qué datos se reciben?
- ¿Qué se debe calcular o mostrar?

2. Descomposición

- dividir el problema en partes pequeñas
- identificar posibles funciones

3. Diseño del algoritmo

- definir estructura general del programa
- decidir qué hace cada función

4. Implementación

- escribir el código en Python

5. Prueba y verificación

- probar con distintos datos
 - validar resultados
-

3. Principio clave

Un problema complejo se resuelve combinando soluciones simples.

Este es el principio central de la programación modular.

4. Ejemplo completo tipo IB

Enunciado

Una escuela almacena las notas de los estudiantes en una lista. Se pide:

- calcular el promedio de cada estudiante
 - determinar si está aprobado (promedio ≥ 7)
 - contar cuántos estudiantes aprobaron
-

5. Descomposición del problema

Podemos dividirlo en tres funciones:

- calcular promedio
 - verificar aprobación
 - contar aprobados
-

6. Implementación modular

Función 1: calcular promedio

```
def promedio(notas):  
    return sum(notas) / len(notas)
```

Función 2: verificar aprobación

```
def aprobado(prom):  
    if prom >= 7:  
        return True
```

```
else:  
    return False
```

Función 3: contar aprobados

```
def contar_aprobados(lista_promedios):  
    contador = 0  
    for p in lista_promedios:  
        if aprobado(p):  
            contador += 1  
    return contador
```

7. Programa principal

```
notas = [  
    [8, 7, 9],  
    [6, 5, 7],  
    [10, 9, 8],  
    [4, 6, 5]  
]  
  
promedios = []  
  
for alumno in notas:  
    prom = promedio(alumno)  
    promedios.append(prom)  
  
resultado = contar_aprobados(promedios)  
print("Cantidad de aprobados:", resultado)
```

8. Flujo del programa

```
Datos de entrada (notas)  
↓  
Función promedio()  
↓  
Lista de promedios  
↓  
Función contar_aprobados()
```

↓
Resultado final

9. Ventajas del diseño modular

- el código es más claro
 - cada función tiene una sola responsabilidad
 - es más fácil detectar errores
 - se puede reutilizar código
 - facilita la resolución de problemas complejos
-

10. Ejercicios propuestos

Ejercicio 1

Sistema de estudiantes:

- calcular promedio
 - determinar aprobado
 - contar aprobados
-

Ejercicio 2

Sistema de ventas:

- total por día
 - mayor venta
 - promedio general
-

Ejercicio 3

Sistema de notas:

- eliminar notas inválidas
 - calcular promedio
 - clasificar estudiantes
-

Ejercicio 4

Diseñar un sistema completo que:

- reciba datos de usuarios
 - procese información
 - genere un informe final
-

CAPÍTULO 4 – Algoritmos

B2.4.1 – Complejidad Algorítmica (Big O)

1. ¿Qué es la complejidad algorítmica?

Cuando escribimos un programa, no basta con que produzca el resultado correcto.

También es importante considerar **cuánto tiempo tarda en ejecutarse y cómo ese tiempo cambia cuando aumenta la cantidad de datos**.

El estudio de este comportamiento se conoce como **complejidad algorítmica**.

La complejidad permite estimar la eficiencia de un algoritmo sin depender del lenguaje de programación o de la velocidad de una computadora.

2. ¿Qué es Big O?

La notación **Big O** describe cómo crece el tiempo de ejecución de un algoritmo a medida que aumenta el tamaño de la entrada.

En otras palabras, responde a la pregunta:

¿Cómo cambia el tiempo de ejecución cuando aumenta la cantidad de datos que debe procesar el algoritmo?

En Big O, el tamaño de la entrada suele representarse con la letra **n**.

3. ¿Por qué no medimos el tiempo en segundos?

Podría parecer lógico medir cuánto tarda un programa en ejecutarse, pero ese tiempo depende de muchos factores:

- velocidad del procesador;
- memoria disponible;
- sistema operativo;

- lenguaje de programación;
- optimizaciones del compilador o intérprete.

Por esta razón, Big O no mide segundos, sino **cómo crece el número de operaciones que realiza un algoritmo**.

4. Concepto de tamaño de entrada

El tamaño de entrada (n) representa la cantidad de datos que recibe un algoritmo.

Por ejemplo:

- una lista con 10 elementos $\rightarrow n = 10$
- una lista con 100 elementos $\rightarrow n = 100$
- una lista con 1000 elementos $\rightarrow n = 1000$

A medida que n aumenta, algunos algoritmos siguen siendo rápidos y otros se vuelven muy lentos.

5. Complejidades más comunes

$O(1)$ – Tiempo constante

El algoritmo siempre realiza la misma cantidad de operaciones, independientemente del tamaño de la entrada.

Ejemplo:

```
numeros = [8, 5, 10, 3]
print(numeros[2])
```

Aunque la lista tenga 10 elementos o 1 millón, acceder a una posición específica requiere una única operación.

$O(n)$ – Tiempo lineal

El tiempo de ejecución crece de forma proporcional al tamaño de la entrada.

Ejemplo:

```
numeros = [4, 8, 2, 7, 5]

for numero in numeros:
    print(numero)
```

Si la lista tiene el doble de elementos, aproximadamente el algoritmo realizará el doble de operaciones.

$O(n^2)$ – Tiempo cuadrático

El algoritmo contiene dos recorridos anidados sobre los mismos datos.

Ejemplo:

```
numeros = [1, 2, 3]

for i in numeros:
    for j in numeros:
        print(i, j)
```

Con tres elementos se realizan nueve iteraciones.

Con diez elementos se realizan cien.

Con cien elementos se realizan diez mil.

El crecimiento es mucho más rápido que en un algoritmo lineal.

$O(\log n)$ – Tiempo logarítmico

En cada paso el algoritmo reduce considerablemente la cantidad de datos que debe analizar.

Un ejemplo clásico es la búsqueda binaria, que estudiaremos más adelante.

Por ejemplo, para buscar un dato en una lista ordenada de 1024 elementos, la búsqueda binaria necesita como máximo unas 10 comparaciones, mientras que una búsqueda lineal podría necesitar hasta 1024.

6. Comparación de crecimiento

Complejidad	Crecimiento	Ejemplo
$O(1)$	Constante	Acceder a un elemento por índice

Complejidad	Crecimiento	Ejemplo
$O(\log n)$	Logarítmico	Búsqueda binaria
$O(n)$	Lineal	Recorrer una lista
$O(n^2)$	Cuadrático	Bubble Sort, Selection Sort

7. Comparación visual

Supongamos listas de distintos tamaños.

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n^2)$
10	1	4	10	100
100	1	7	100	10000
1000	1	10	1000	1000000

Puede observarse que la diferencia entre $O(n)$ y $O(n^2)$ se vuelve enorme cuando aumenta la cantidad de datos.

8. ¿Cómo identificar la complejidad de un algoritmo?

Una forma sencilla es observar la estructura del código.

Sin bucles

```
x = lista[3]
```

Complejidad:

$O(1)$

Un único bucle

```
for elemento in lista:
    print(elemento)
```

Complejidad:

$O(n)$

Dos bucles anidados

```
for i in lista:
    for j in lista:
        print(i, j)
```

Complejidad:

$O(n^2)$

9. ¿Por qué es importante Big O?

Elegir un algoritmo eficiente puede marcar una gran diferencia cuando se trabaja con grandes cantidades de datos.

Por ejemplo:

- buscar un estudiante entre 20 alumnos puede ser muy rápido con cualquier algoritmo;
- buscar un estudiante entre un millón de registros requiere algoritmos mucho más eficientes.

Por eso, en informática no solo interesa obtener el resultado correcto, sino hacerlo utilizando la menor cantidad posible de recursos.

10. Ejercicios

Ejercicio 1

Indicar la complejidad temporal de los siguientes fragmentos de código.

```
for i in lista:
    print(i)
```

Ejercicio 2

Indicar la complejidad del siguiente algoritmo.

```
for i in lista:
    for j in lista:
        print(i, j)
```

Ejercicio 3

Explicar por qué acceder a un elemento mediante su índice tiene complejidad **$O(1)$** .

Ejercicio 4

Completar la siguiente tabla indicando cuál algoritmo sería más eficiente para listas muy grandes.

Complejidad	¿Más eficiente?
$O(1)$	
$O(\log n)$	
$O(n)$	
$O(n^2)$	

Ejercicio 5

Un algoritmo recorre una lista de **5000** elementos una única vez.

1. ¿Cuál es su complejidad temporal?
 2. ¿Qué ocurriría si la lista creciera a **10000** elementos?
 3. Comparar este algoritmo con otro de complejidad **$O(n^2)$** para la misma cantidad de datos.
-

B2.4.2 – Búsqueda lineal (Linear Search)

1. ¿Qué es un algoritmo de búsqueda?

Un algoritmo de búsqueda es un procedimiento que permite determinar si un elemento se encuentra dentro de una colección de datos.

En programación, buscar información es una tarea muy frecuente.

Por ejemplo:

- buscar un alumno por su nombre;
- buscar un producto por su código;
- buscar una palabra en un documento;
- buscar un número dentro de una lista.

Existen distintos algoritmos de búsqueda, y cada uno presenta ventajas y limitaciones dependiendo de la organización de los datos.

2. ¿Qué es la búsqueda lineal?

La búsqueda lineal consiste en recorrer los elementos de una colección uno por uno hasta encontrar el valor buscado o hasta llegar al final de la colección.

No requiere que los datos estén ordenados.

Es el algoritmo de búsqueda más simple de implementar.

3. Funcionamiento paso a paso

Supongamos la siguiente lista:

```
numeros = [8, 3, 12, 5, 9, 15]
```

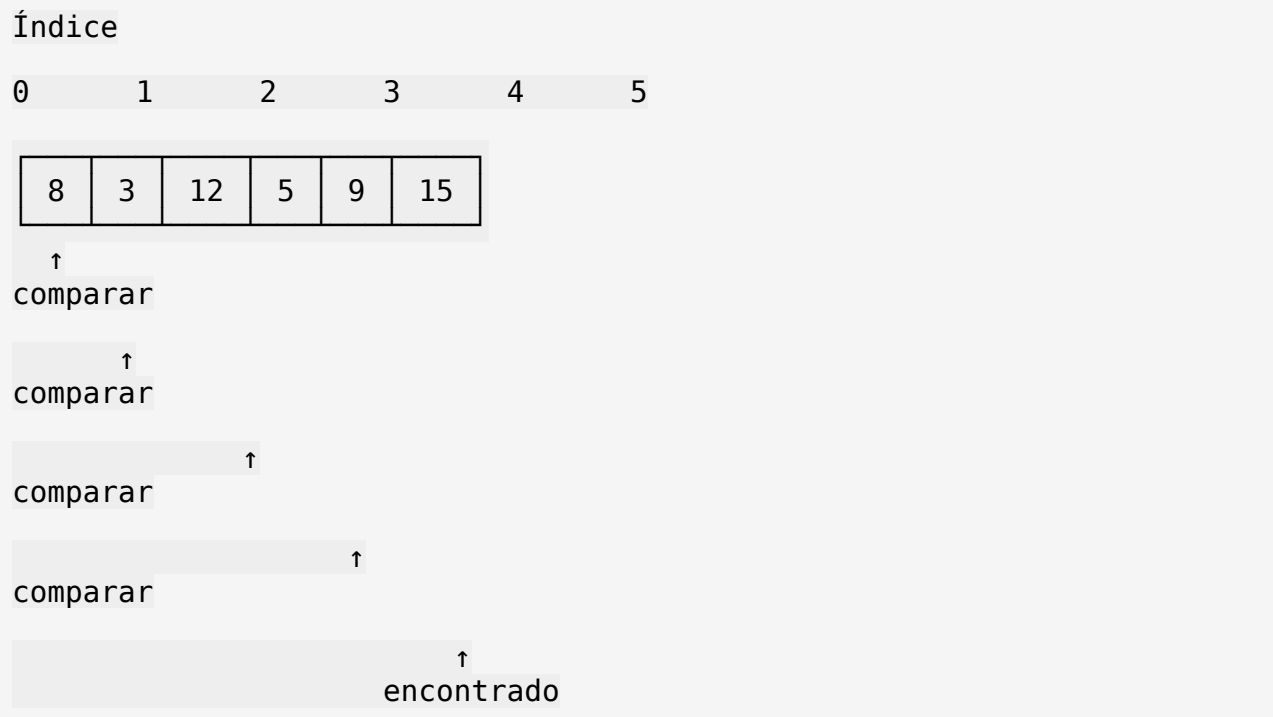
Se desea buscar el número 9.

El algoritmo compara:

```
8 → no coincide  
3 → no coincide  
12 → no coincide  
5 → no coincide  
9 → encontrado
```

La búsqueda finaliza en el momento en que encuentra el elemento.

4. Representación gráfica



5. Implementación en Python

Buscar un valor

```
numeros = [8, 3, 12, 5, 9, 15]
buscado = 9
encontrado = False
for numero in numeros:
    if numero == buscado:
        encontrado = True
        break
print(encontrado)
```

Salida:

True

6. ¿Qué hace la instrucción `break`?

La instrucción `break` finaliza inmediatamente el bucle.

En este caso evita seguir recorriendo la lista cuando el elemento ya fue encontrado.

Sin `break`, el algoritmo seguiría comparando los elementos restantes aunque ya conozca la respuesta.

Ejemplo

```
for numero in numeros:
    if numero == buscado:
        print("Encontrado")
        break
```

7. Buscar la posición de un elemento

Muchas veces no solo interesa saber si un elemento existe, sino también conocer su posición.

```
numeros = [8, 3, 12, 5, 9, 15]
buscado = 9
for i in range(len(numeros)):
    if numeros[i] == buscado:
        print("Posición:", i)
        break
```

Salida:

```
Posición: 4
```

8. ¿Qué ocurre si el elemento no existe?

```
numeros = [8, 3, 12, 5, 9]
buscado = 20
encontrado = False
for numero in numeros:
    if numero == buscado:
        encontrado = True
        break
if encontrado:
    print("Elemento encontrado")
else:
    print("Elemento no encontrado")
```

Salida:

Elemento no encontrado

9. Complejidad temporal

La búsqueda lineal puede recorrer toda la lista.

Por eso su complejidad es:

$O(n)$

donde **n** representa la cantidad de elementos de la colección.

Mejor caso

El elemento está en la primera posición.

Una comparación

Caso promedio

El elemento está aproximadamente en la mitad.

$n / 2$ comparaciones

Peor caso

El elemento está al final o no existe.

n comparaciones

10. Ventajas

- Fácil de comprender.
 - Muy sencilla de implementar.
 - Funciona con listas ordenadas y desordenadas.
 - No requiere preparación previa de los datos.
-

11. Desventajas

- Puede resultar lenta para listas muy grandes.
 - Recorre los datos de forma secuencial.
 - Existen algoritmos más eficientes cuando la lista está ordenada.
-

12. Comparación con otros algoritmos

Algoritmo	Lista ordenada	Complejidad
Búsqueda lineal	No es necesaria	$O(n)$
Búsqueda binaria	Sí	$O(\log n)$

La búsqueda binaria será estudiada en la siguiente sección.

13. Ejercicios

Ejercicio 1

Escribir un programa que determine si un número ingresado por el usuario se encuentra en una lista.

Ejercicio 2

Modificar el programa anterior para mostrar la posición del elemento encontrado.

Ejercicio 3

Contar cuántas comparaciones realiza la búsqueda para encontrar un elemento determinado.

Ejercicio 4

Explicar cuál sería el mejor caso y el peor caso de la búsqueda lineal.

Ejercicio 5

Una escuela almacena el número de identificación de sus estudiantes en una lista.

Desarrollar un programa modular que permita:

- buscar un estudiante por su identificación;
 - indicar si fue encontrado;
 - mostrar la posición que ocupa en la lista;
 - informar cuántas comparaciones realizó el algoritmo antes de finalizar la búsqueda.
-

B2.4.3 – Búsqueda binaria (Binary Search)

1. ¿Qué es la búsqueda binaria?

La búsqueda binaria es un algoritmo que permite buscar un elemento dentro de una lista **ordenada**.

A diferencia de la búsqueda lineal, no revisa los elementos uno por uno. En cada paso, compara el elemento buscado con el elemento ubicado en el centro de la lista y descarta la mitad de los datos que no pueden contener el valor buscado.

Gracias a esta estrategia, la búsqueda binaria es considerablemente más rápida cuando se trabaja con grandes cantidades de datos.

2. Condición fundamental

La búsqueda binaria **solo funciona correctamente si la lista está ordenada**.

Por ejemplo, la siguiente lista puede utilizarse:

```
numeros = [3, 5, 8, 12, 15, 20, 27, 31]
```

En cambio, esta lista no puede utilizarse:

```
numeros = [12, 3, 27, 5, 20, 8, 31, 15]
```

Si la lista no está ordenada, el algoritmo puede producir resultados incorrectos.

3. ¿Cómo funciona?

Supongamos la siguiente lista ordenada:

Índice							
0	1	2	3	4	5	6	7
3	5	8	12	15	20	27	31

Queremos buscar el número **20**.

Paso 1

Observamos el elemento central.

3	5	8	[12]	15	20	27	31
			↑				

Como **20 es mayor que 12**, descartamos toda la mitad izquierda.

Paso 2

Ahora solo queda analizar:

```
15 20 27 31
```

El nuevo elemento central es:

```
15 [20] 27 31
    ↑
```

El valor buscado fue encontrado.

Solo fueron necesarias **dos comparaciones**.

4. Algoritmo paso a paso

1. Ordenar la lista (si aún no lo está).
 2. Calcular el índice central.
 3. Comparar el elemento central con el valor buscado.
 4. Si son iguales, finalizar.
 5. Si el valor buscado es menor, continuar únicamente con la mitad izquierda.
 6. Si el valor buscado es mayor, continuar únicamente con la mitad derecha.
 7. Repetir el proceso hasta encontrar el elemento o hasta que ya no queden elementos por analizar.
-

5. Variables utilizadas

Durante la búsqueda binaria se utilizan tres índices principales:

- **inicio:** primer índice del intervalo de búsqueda.
- **fin:** último índice del intervalo de búsqueda.
- **medio:** índice ubicado en el centro del intervalo.

Inicialmente:

```
inicio = 0
fin = len(lista) - 1
```

El índice medio se calcula mediante:

```
medio = (inicio + fin) // 2
```

Se utiliza `//` porque el índice debe ser un número entero.

6. Implementación en Python

```
numeros = [3, 5, 8, 12, 15, 20, 27, 31]
buscado = 20

inicio = 0
fin = len(numeros) - 1

encontrado = False

while inicio <= fin:
    medio = (inicio + fin) // 2

    if numeros[medio] == buscado:
        encontrado = True
        break

    elif buscado < numeros[medio]:
        fin = medio - 1

    else:
        inicio = medio + 1

print(encontrado)
```

Salida:

```
True
```

7. Obtener la posición del elemento

En muchas ocasiones interesa conocer el índice donde se encuentra el valor.

```
numeros = [3, 5, 8, 12, 15, 20, 27, 31]
buscado = 20

inicio = 0
fin = len(numeros) - 1
```

```
while inicio <= fin:
    medio = (inicio + fin) // 2

    if numeros[medio] == buscado:
        print("Posición:", medio)
        break

    elif buscado < numeros[medio]:
        fin = medio - 1

    else:
        inicio = medio + 1
```

Salida:

Posición: 5

8. ¿Qué ocurre si el elemento no existe?

Si el algoritmo termina el bucle sin encontrar el elemento, significa que el valor no pertenece a la lista.

```
if encontrado:
    print("Elemento encontrado")
else:
    print("Elemento no encontrado")
```

9. Complejidad temporal

En cada iteración, la búsqueda binaria elimina aproximadamente la mitad de los elementos restantes.

Por este motivo su complejidad temporal es:

$O(\log n)$

Esto significa que el número de comparaciones aumenta muy lentamente aunque la cantidad de datos crezca considerablemente.

10. Comparación con la búsqueda lineal

Supongamos una lista de **1 000 000** de elementos.

Algoritmo	Comparaciones máximas
Búsqueda lineal	1 000 000
Búsqueda binaria	Aproximadamente 20

Esta diferencia explica por qué la búsqueda binaria es uno de los algoritmos de búsqueda más utilizados.

11. Ventajas

- Mucho más rápida que la búsqueda lineal para listas grandes.
 - Reduce el número de comparaciones en cada paso.
 - Presenta complejidad **$O(\log n)$** .
-

12. Desventajas

- Solo puede utilizarse sobre listas ordenadas.
 - Su implementación es más compleja que la búsqueda lineal.
 - Si la lista cambia constantemente, mantenerla ordenada puede resultar costoso.
-

13. Comparación entre ambos algoritmos

Característica	Búsqueda lineal	Búsqueda binaria
Requiere lista ordenada	No	Sí
Complejidad temporal	$O(n)$	$O(\log n)$
Facilidad de implementación	Muy sencilla	Moderada
Eficiencia en listas grandes	Baja	Muy alta

14. Ejercicios

Ejercicio 1

Implementar una búsqueda binaria que indique si un número pertenece a una lista ordenada.

Ejercicio 2

Modificar el algoritmo para mostrar la posición donde se encuentra el elemento.

Ejercicio 3

Explicar por qué la búsqueda binaria no funciona correctamente sobre una lista desordenada.

Ejercicio 4

Comparar la cantidad máxima de comparaciones necesarias para buscar un elemento en una lista de 500 elementos utilizando:

- búsqueda lineal;
 - búsqueda binaria.
-

Ejercicio 5

Una biblioteca almacena los códigos de sus libros en una lista ordenada.

Desarrollar un programa modular que permita:

- buscar un código utilizando búsqueda binaria;
 - indicar si el libro existe;
 - mostrar la posición donde fue encontrado;
 - contar la cantidad de iteraciones realizadas por el algoritmo;
 - comparar el número de comparaciones con una búsqueda lineal sobre la misma lista.
-

B2.4.4 – Bubble Sort (Ordenamiento por burbuja)

1. ¿Qué es un algoritmo de ordenamiento?

Un algoritmo de ordenamiento es un procedimiento que reorganiza los elementos de una colección siguiendo un criterio determinado.

El criterio más común es el orden:

- Ascendente (de menor a mayor).
- Descendente (de mayor a menor).

Muchos algoritmos requieren que los datos estén ordenados antes de poder utilizarlos de forma eficiente. Por ejemplo, la búsqueda binaria solo funciona correctamente sobre listas ordenadas.

2. ¿Qué es Bubble Sort?

Bubble Sort (ordenamiento por burbuja) es uno de los algoritmos de ordenamiento más sencillos de comprender e implementar.

Su funcionamiento consiste en comparar elementos **adyacentes** de una lista e intercambiarlos cuando se encuentran en el orden incorrecto.

Este proceso se repite varias veces hasta que toda la lista queda ordenada.

El nombre "burbuja" proviene de la idea de que, en cada recorrido, el elemento de mayor valor "sube" hacia el final de la lista, de forma similar a una burbuja que asciende en el agua.

3. Funcionamiento paso a paso

Supongamos la siguiente lista:

```
numeros = [7, 3, 5, 2]
```

Queremos ordenarla de menor a mayor.

Primera pasada

Se comparan los dos primeros elementos.

```
[7, 3, 5, 2]
```

```
7 > 3
```

Se intercambian.

Resultado:

[3, 7, 5, 2]

Ahora se comparan:

7 y 5

Como 7 es mayor, se intercambian.

[3, 5, 7, 2]

Se comparan:

7 y 2

Resultado:

[3, 5, 2, 7]

El número 7 ya quedó en su posición definitiva.

Segunda pasada

Ahora solo es necesario recorrer hasta el penúltimo elemento.

[3, 5, 2, 7]

Comparaciones:

3 y 5 → correctos

5 y 2 → intercambio

Resultado:

[3, 2, 5, 7]

Tercera pasada

3 y 2 → intercambio

Resultado:

[2, 3, 5, 7]

La lista ya quedó ordenada.

4. Idea fundamental

Bubble Sort compara siempre elementos consecutivos.

Si están en el orden incorrecto:

→ los intercambia.

Este proceso continúa hasta que ya no sea necesario realizar intercambios.

5. Intercambio de valores en Python

En Python el intercambio puede realizarse de forma muy sencilla.

```
a = 10
b = 20

a, b = b, a

print(a, b)
```

Salida:

20 10

Esta característica hace que Bubble Sort sea más fácil de implementar que en otros lenguajes.

6. Implementación en Python

```
numeros = [7, 3, 5, 2]
n = len(numeros)
for i in range(n - 1):
    for j in range(n - 1 - i):
        if numeros[j] > numeros[j + 1]:
            numeros[j], numeros[j + 1] = numeros[j + 1], numeros[j]
print(numeros)
```

Salida:

```
[2, 3, 5, 7]
```

7. ¿Por qué aparecen dos bucles?

Bubble Sort necesita recorrer la lista varias veces.

- El bucle externo controla la cantidad de pasadas.
- El bucle interno realiza las comparaciones e intercambios.

En cada pasada, el mayor elemento pendiente queda ubicado en su posición definitiva.

Por este motivo, en cada nueva pasada es necesario recorrer un elemento menos.

8. Optimización del algoritmo

Si durante una pasada no se realiza ningún intercambio, significa que la lista ya está ordenada.

En ese caso, el algoritmo puede finalizar antes.

```
intercambio = False
```

Cada vez que se intercambian dos elementos:

```
intercambio = True
```

Al finalizar la pasada:

```
if not intercambio:  
    break
```

Esta optimización mejora el rendimiento cuando la lista ya está ordenada o casi ordenada.

9. Complejidad temporal

Mejor caso

La lista ya está ordenada.

Con la optimización:

$O(n)$

Caso promedio

Se realizan numerosas comparaciones e intercambios.

$O(n^2)$

Peor caso

La lista está completamente desordenada.

$O(n^2)$

10. Ventajas

- Fácil de comprender.
 - Fácil de implementar.
 - Adecuado para listas pequeñas.
 - Muy útil para aprender algoritmos de ordenamiento.
-

11. Desventajas

- Poco eficiente para listas grandes.
 - Realiza muchas comparaciones innecesarias.
 - Existen algoritmos considerablemente más rápidos.
-

12. Ejemplo de comparación

Supongamos una lista con 1000 elementos.

Algoritmo	Complejidad
Bubble Sort	$O(n^2)$
Búsqueda lineal	$O(n)$
Búsqueda binaria	$O(\log n)$

Puede observarse que ordenar una lista requiere más trabajo que simplemente recorrerla o buscar un elemento.

13. Ejercicios

Ejercicio 1

Aplicar Bubble Sort para ordenar la siguiente lista:

[12, 5, 9, 3, 15, 8]

Ejercicio 2

Mostrar el contenido de la lista después de cada pasada del algoritmo.

Ejercicio 3

Modificar Bubble Sort para ordenar una lista en orden descendente.

Ejercicio 4

Explicar por qué Bubble Sort necesita varias pasadas para ordenar completamente una lista.

Ejercicio 5

Una escuela almacena las calificaciones finales de sus estudiantes en una lista.

Desarrollar un programa modular que:

- ordene las calificaciones utilizando Bubble Sort;
 - muestre la lista antes y después del ordenamiento;
 - cuente cuántos intercambios se realizaron durante el proceso;
 - indique la complejidad temporal del algoritmo y explique por qué.
-

B2.4.5 – Selection Sort (Ordenamiento por selección)

1. ¿Qué es Selection Sort?

Selection Sort (ordenamiento por selección) es un algoritmo que ordena una lista buscando, en cada pasada, el elemento más pequeño de la parte desordenada de la lista y colocándolo en la posición correcta.

A diferencia de Bubble Sort, **no intercambia elementos continuamente**, sino que realiza un único intercambio al finalizar cada pasada.

2. Idea principal

Supongamos la siguiente lista:

```
numeros = [7, 3, 5, 2]
```

La idea es:

1. Buscar el menor elemento.
2. Colocarlo en la primera posición.
3. Buscar el menor de los elementos restantes.

- Colocar en la segunda posición.
 - Repetir el proceso hasta ordenar toda la lista.
-

3. Funcionamiento paso a paso

Lista inicial:

[7, 3, 5, 2]

Primera pasada

Se busca el menor elemento.

7 3 5 2
 ↑
 menor

El menor es **2**.

Se intercambia con el primer elemento.

[2, 3, 5, 7]

Segunda pasada

Ahora solo se analiza la parte restante.

2 | 3 5 7

El menor ya es **3**.

No es necesario intercambiar.

Tercera pasada

2 3 | 5 7

El menor es **5**.

No se realiza intercambio.

Cuarta pasada

Solo queda un elemento.

La lista ya está ordenada.

4. Representación gráfica

Lista original

[7, 3, 5, 2]

↓

Buscar menor

↓

Intercambiar

↓

[2, 3, 5, 7]

↓

Repetir sobre el resto de la lista

5. Algoritmo

Para cada posición:

- asumir que el primer elemento es el menor;
 - recorrer el resto de la lista buscando un elemento menor;
 - actualizar la posición del menor cuando sea necesario;
 - realizar un único intercambio.
-

6. Implementación en Python

```
numeros = [7, 3, 5, 2]
n = len(numeros)
for i in range(n - 1):
    menor = i
    for j in range(i + 1, n):
        if numeros[j] < numeros[menor]:
            menor = j
    numeros[i], numeros[menor] = numeros[menor], numeros[i]
print(numeros)
```

Salida:

```
[2, 3, 5, 7]
```

7. Explicación del algoritmo

En cada iteración del bucle externo:

- `i` representa la posición que debe ordenarse.

La variable:

```
menor = i
```

supone inicialmente que el primer elemento es el menor.

Luego, el bucle interno recorre el resto de la lista.

Si encuentra un valor menor:

```
menor = j
```

actualiza la posición del nuevo mínimo.

Al finalizar el recorrido:

```
numeros[i], numeros[menor] = numeros[menor], numeros[i]
```

coloca definitivamente el menor elemento en su posición correcta.

8. Ejemplo paso a paso

Lista:

[10, 4, 8, 2]

Primera pasada

Menor:

2

Resultado:

[2, 4, 8, 10]

Segunda pasada

Sublista:

[4, 8, 10]

El menor ya es 4.

No hay intercambio.

Tercera pasada

Sublista:

[8, 10]

Ya está ordenada.

9. Complejidad temporal

Mejor caso

Aunque la lista ya esté ordenada, Selection Sort debe recorrer igualmente toda la parte restante para comprobar cuál es el menor.

$O(n^2)$

Caso promedio

$O(n^2)$

Peor caso

$O(n^2)$

10. Cantidad de intercambios

Una característica importante de Selection Sort es que realiza pocos intercambios.

En el peor caso realiza aproximadamente:

$n - 1$ intercambios

Esto es mucho menos que Bubble Sort.

11. Comparación con Bubble Sort

Característica	Bubble Sort	Selection Sort
Estrategia	Intercambia elementos vecinos	Busca el menor elemento
Comparaciones	Muchas	Muchas
Intercambios	Muchos	Pocos
Mejor caso	$O(n)$ (con optimización)	$O(n^2)$
Caso promedio	$O(n^2)$	$O(n^2)$
Peor caso	$O(n^2)$	$O(n^2)$

12. Ventajas

- Fácil de comprender.
 - Implementación sencilla.
 - Realiza pocos intercambios.
 - Útil para introducir algoritmos de ordenamiento.
-

13. Desventajas

- Poco eficiente para listas grandes.
 - Siempre recorre la parte restante de la lista.
 - Existen algoritmos considerablemente más rápidos, como Merge Sort o Quick Sort (aunque estos no forman parte del contenido obligatorio de este curso).
-

14. Ejercicios

Ejercicio 1

Ordenar mediante Selection Sort la siguiente lista:

[18, 7, 12, 4, 9]

Ejercicio 2

Mostrar el contenido de la lista después de cada pasada del algoritmo.

Ejercicio 3

Modificar el algoritmo para ordenar la lista de mayor a menor.

Ejercicio 4

Comparar Bubble Sort y Selection Sort indicando:

- diferencias en la estrategia;
- cantidad de intercambios;
- complejidad temporal;
- situaciones donde podría preferirse uno sobre el otro.

Ejercicio 5

Una empresa almacena las edades de sus empleados en una lista.

Desarrollar un programa modular que:

- ordene las edades utilizando Selection Sort;
 - muestre la lista antes y después del ordenamiento;
 - cuente cuántas comparaciones y cuántos intercambios realizó el algoritmo;
 - compare esos resultados con Bubble Sort utilizando la misma lista.
-

B2.4.6 – Comparación de algoritmos

1. ¿Por qué existen distintos algoritmos?

En programación, un mismo problema puede resolverse de diferentes maneras.

Por ejemplo, si queremos buscar un estudiante dentro de una lista, podemos utilizar:

- Búsqueda lineal.
- Búsqueda binaria.

Ambos algoritmos obtienen el mismo resultado, pero no tienen la misma eficiencia.

Del mismo modo, una lista puede ordenarse mediante distintos algoritmos, como Bubble Sort o Selection Sort.

La elección del algoritmo dependerá de las características del problema y de los datos disponibles.

2. Comparación de algoritmos de búsqueda

Los dos algoritmos de búsqueda estudiados presentan diferencias importantes.

Característica	Búsqueda lineal	Búsqueda binaria
Lista ordenada	No es necesaria	Obligatoria
Complejidad	$O(n)$	$O(\log n)$
Facilidad de implementación	Muy sencilla	Moderada

Característica	Búsqueda lineal	Búsqueda binaria
Velocidad en listas pequeñas	Buena	Buena
Velocidad en listas grandes	Baja	Muy alta

¿Cuándo utilizar búsqueda lineal?

La búsqueda lineal es recomendable cuando:

- la lista es pequeña;
- los datos no están ordenados;
- la lista cambia constantemente;
- ordenar la lista no resulta conveniente.

Ejemplos:

- Buscar un nombre en una lista de 20 alumnos.
 - Buscar un producto recién agregado a un inventario.
-

¿Cuándo utilizar búsqueda binaria?

La búsqueda binaria es recomendable cuando:

- la lista ya está ordenada;
- la colección contiene una gran cantidad de datos;
- se realizarán muchas búsquedas sobre la misma lista.

Ejemplos:

- Buscar un cliente entre un millón de registros.
 - Buscar un libro en el catálogo de una biblioteca ordenado por código.
-

3. Comparación de algoritmos de ordenamiento

Los algoritmos Bubble Sort y Selection Sort también presentan diferencias.

Característica	Bubble Sort	Selection Sort
Estrategia	Intercambia elementos consecutivos	Busca el menor elemento
Intercambios	Muchos	Pocos
Mejor caso	$O(n)$ (optimizado)	$O(n^2)$
Caso promedio	$O(n^2)$	$O(n^2)$

Característica	Bubble Sort	Selection Sort
Peor caso	$O(n^2)$	$O(n^2)$
Facilidad de comprensión	Muy alta	Alta

4. Bubble Sort vs Selection Sort

Aunque ambos tienen complejidad $O(n^2)$, su comportamiento es diferente.

Bubble Sort

- compara elementos consecutivos;
- puede detenerse antes si la lista ya está ordenada (versión optimizada);
- realiza muchos intercambios.

Selection Sort

- busca el menor elemento en cada pasada;
- realiza un solo intercambio por pasada;
- siempre recorre la parte restante de la lista.

5. Comparación de complejidad

Algoritmo	Complejidad
Acceso por índice	$O(1)$
Búsqueda binaria	$O(\log n)$
Búsqueda lineal	$O(n)$
Bubble Sort	$O(n^2)$
Selection Sort	$O(n^2)$

En general:

$$O(1) < O(\log n) < O(n) < O(n^2)$$

Cuanto menor sea la complejidad, más eficiente será el algoritmo para grandes cantidades de datos.

6. Ejemplo comparativo

Supongamos una lista con **1000 elementos**.

Algoritmo	Operaciones aproximadas
Acceso por índice	1
Búsqueda binaria	10
Búsqueda lineal	1000
Bubble Sort	1 000 000
Selection Sort	1 000 000

Este ejemplo muestra cómo la eficiencia de un algoritmo cobra importancia a medida que aumenta el tamaño de la entrada.

7. ¿Cómo elegir un algoritmo?

Antes de programar, es recomendable responder las siguientes preguntas:

1. ¿La lista está ordenada?
2. ¿Cuántos elementos contiene?
3. ¿Se realizarán muchas búsquedas?
4. ¿Es necesario ordenar los datos?
5. ¿Qué es más importante: rapidez, simplicidad o facilidad de mantenimiento?

Responder estas preguntas ayuda a seleccionar el algoritmo más adecuado.

8. Casos prácticos

Caso 1

Una escuela almacena las notas de **25 alumnos**.

¿Qué algoritmo de búsqueda utilizarías?

Respuesta: Búsqueda lineal.

Justificación: la lista es pequeña y la diferencia de rendimiento es mínima.

Caso 2

Una empresa posee una base de datos con **5 millones de clientes**, ordenados por número de documento.

¿Qué algoritmo utilizarías?

Respuesta: Búsqueda binaria.

Justificación: la lista ya está ordenada y la búsqueda binaria requiere muchas menos comparaciones.

Caso 3

Una lista contiene únicamente diez números desordenados.

¿Qué algoritmo de ordenamiento utilizarías?

Respuesta: Bubble Sort o Selection Sort.

Justificación: ambos son sencillos y el tamaño reducido de la lista hace que la diferencia de rendimiento sea poco significativa.

Caso 4

Una lista ya se encuentra casi completamente ordenada.

¿Qué algoritmo sería más conveniente?

Respuesta: Bubble Sort optimizado.

Justificación: puede detectar que no son necesarios más intercambios y finalizar antes.

9. Ejercicios

Ejercicio 1

Completar la siguiente tabla:

Situación	Algoritmo recomendado	Justificación
Buscar un alumno en una lista desordenada de 30 estudiantes		
Buscar un libro en una biblioteca ordenada		
Ordenar una lista de 15 números		

Ejercicio 2

Explicar las principales diferencias entre Bubble Sort y Selection Sort.

Ejercicio 3

Comparar la búsqueda lineal y la búsqueda binaria indicando:

- requisitos;
 - complejidad;
 - ventajas;
 - desventajas.
-

Ejercicio 4

Ordenar una lista utilizando Bubble Sort y luego buscar un elemento mediante búsqueda binaria. Explicar por qué fue necesario ordenar la lista antes de realizar la búsqueda.

Ejercicio 5

Una empresa almacena la información de **200 000 productos** en una base de datos.

El sistema debe permitir:

- buscar productos por código;
- ordenar los productos por precio;
- realizar miles de búsquedas diarias.

Responder:

1. ¿Qué algoritmo de búsqueda utilizarías? Justificar.
 2. ¿Qué algoritmo de ordenamiento de los estudiados utilizarías? Justificar.
 3. Explicar por qué la elección de un algoritmo eficiente resulta importante en este caso.
-

CAPÍTULO 5 – Archivos

B2.5.1 – Apertura y cierre de archivos

1. ¿Qué es un archivo?

Un **archivo** es un conjunto de datos almacenados de forma permanente en un dispositivo de almacenamiento (disco rígido, SSD, memoria USB, etc.).

A diferencia de las variables y las listas, cuyo contenido se pierde cuando el programa finaliza, los archivos permiten conservar la información para utilizarla posteriormente.

Ejemplos de archivos son:

- documentos de texto (.txt)
- hojas de cálculo (.csv, .xlsx)
- imágenes (.jpg, .png)
- archivos PDF (.pdf)
- bases de datos

En este capítulo trabajaremos principalmente con **archivos de texto (.txt)**.

2. ¿Por qué utilizar archivos?

Hasta ahora nuestros programas utilizaban datos como estos:

```
notas = [8, 6, 9, 10]
```

Cuando el programa termina, esa información desaparece de la memoria.

Si queremos conservarla, debemos guardarla en un archivo.

Por ejemplo:

```
notas.txt
```

```
8  
6
```

9
10

La próxima vez que ejecutemos el programa podremos volver a leer esos datos.

3. Memoria RAM vs almacenamiento permanente

Es importante distinguir estos dos conceptos.

Memoria RAM	Almacenamiento permanente
Temporal	Permanente
Se borra al apagar el equipo	Permanece guardado
Muy rápida	Más lenta
Variables, listas y objetos	Archivos

4. Abrir un archivo

En Python se utiliza la función:

```
open()
```

Su sintaxis general es:

```
archivo = open("nombre_del_archivo", "modo")
```

Por ejemplo:

```
archivo = open("datos.txt", "r")
```

Aquí:

- `"datos.txt"` es el nombre del archivo.
 - `"r"` indica el modo de apertura.
-

5. Modos de apertura

Python permite abrir un archivo de diferentes maneras.

Modo

o Significado

"r"	Leer un archivo existente
"w"	Escribir (crea el archivo o sobrescribe el contenido existente)
"a"	Agregar información al final del archivo
"x"	Crear un archivo nuevo. Produce un error si ya existe.

Durante este curso utilizaremos principalmente los modos **r**, **w** y **a**.

6. Abrir un archivo para lectura

```
archivo = open("notas.txt", "r")
```

En este momento el archivo está abierto y el programa puede leer su contenido.

Sin embargo, todavía no se ha leído ninguna información.

7. Cerrar un archivo

Cuando ya no es necesario utilizar el archivo, debe cerrarse mediante:

```
archivo.close()
```

Ejemplo:

```
archivo = open("notas.txt", "r")
archivo.close()
```

Cerrar el archivo libera los recursos utilizados por el sistema operativo.

8. ¿Por qué es importante cerrar un archivo?

Si un archivo permanece abierto:

- consume recursos del sistema;
- otros programas podrían no poder acceder a él;
- existe mayor riesgo de pérdida de información si el programa finaliza inesperadamente.

Por este motivo, siempre debe cerrarse cuando deja de utilizarse.

9. La estructura `with`

Python ofrece una forma más segura y recomendada de trabajar con archivos.

En lugar de utilizar:

```
archivo = open("notas.txt", "r")  
  
# operaciones  
  
archivo.close()
```

se utiliza:

```
with open("notas.txt", "r") as archivo:  
    # operaciones
```

Cuando termina el bloque `with`, Python cierra automáticamente el archivo, incluso si ocurre un error durante la ejecución.

Por esta razón, **esta es la forma recomendada de trabajar con archivos en Python**.

10. Comparación entre ambas formas

Forma tradicional

```
archivo = open("datos.txt", "r")  
  
# trabajar con el archivo  
  
archivo.close()
```

Ventaja:

- sencilla de comprender.

Desventaja:

- el programador debe recordar cerrar el archivo.
-

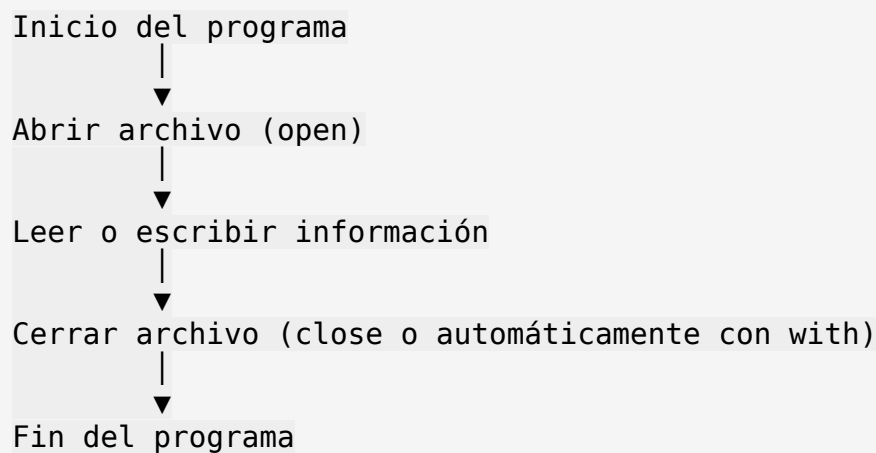
Forma recomendada

```
with open("datos.txt", "r") as archivo:  
    # trabajar con el archivo
```

Ventajas:

- el archivo se cierra automáticamente;
 - el código es más seguro;
 - evita errores frecuentes.
-

11. Flujo de trabajo con archivos



12. Ejercicios

Ejercicio 1

Explicar la diferencia entre la memoria RAM y el almacenamiento permanente.

Ejercicio 2

Indicar qué modo (r, w o a) utilizarías en las siguientes situaciones:

- a) Leer una lista de estudiantes guardada en un archivo.
 - b) Crear un nuevo archivo con las notas finales.
 - c) Agregar un nuevo estudiante a un archivo existente.
-

Ejercicio 3

Escribir un programa que abra un archivo llamado `datos.txt` utilizando:

- la forma tradicional (`open()` y `close()`);
- la estructura `with`.

(No es necesario leer ni escribir información todavía).

Ejercicio 4

Explicar por qué se recomienda utilizar `with` en lugar de `open()` y `close()`.

Ejercicio 5

Una escuela necesita desarrollar un sistema para almacenar la información de sus estudiantes.

Antes de comenzar a programar, responder:

1. ¿Por qué es conveniente utilizar archivos en lugar de listas para almacenar la información?
 2. ¿Qué modo de apertura utilizarías para:
 - consultar estudiantes?
 - registrar nuevos estudiantes?
 - actualizar completamente el archivo?
 3. ¿Qué ventajas ofrece el uso de la estructura `with` en este sistema?
-

B2.5.2 – Lectura de archivos

1. ¿Qué significa leer un archivo?

Leer un archivo consiste en recuperar la información almacenada en él para que el programa pueda utilizarla.

Supongamos que existe el siguiente archivo llamado **notas.txt**:

```
8
6
9
10
7
```

Nuestro programa puede abrir este archivo, leer esos datos y almacenarlos en variables o listas para procesarlos.

2. Abrir un archivo para lectura

Como vimos en la sección anterior, primero debemos abrir el archivo utilizando el modo `"r"`.

```
with open("notas.txt", "r") as archivo:
    # leer información
```

A partir de este momento, podemos acceder a su contenido.

3. Método `read()`

El método `read()` lee **todo el contenido del archivo** y lo devuelve como una única cadena de texto (`str`).

Ejemplo

Archivo:

```
Juan
María
Pedro
Ana
```

Código:

```
with open("alumnos.txt", "r") as archivo:
    contenido = archivo.read()
print(contenido)
```

Salida:

```
Juan
María
Pedro
Ana
```

4. Características de `read()`

- Lee el archivo completo.
 - Devuelve una única cadena de texto.
 - Conserva los saltos de línea (`\n`).
 - Es útil cuando el archivo es pequeño.
-

5. Método `readline()`

`readline()` lee **una sola línea** del archivo cada vez que se ejecuta.

Ejemplo:

```
with open("alumnos.txt", "r") as archivo:
    linea1 = archivo.readline()
    linea2 = archivo.readline()
print(linea1)
print(linea2)
```

Salida:

```
Juan
María
```

Cada llamada a `readline()` avanza automáticamente a la siguiente línea.

6. Método `readlines()`

Este método lee todas las líneas del archivo y las almacena en una lista.

Ejemplo:

```
with open("alumnos.txt", "r") as archivo:
    lineas = archivo.readlines()
print(lineas)
```

Salida:

```
['Juan\n', 'María\n', 'Pedro\n', 'Ana\n']
```

Cada elemento de la lista corresponde a una línea del archivo.

7. El carácter `\n`

Cuando leemos un archivo, normalmente aparece el carácter especial:

```
\n
```

Este representa un **salto de línea**.

Por ejemplo:

```
"Juan\n"
```

significa:

```
Juan
```

seguido de un salto de línea.

8. Eliminar el salto de línea (`strip()`)

En muchas ocasiones resulta conveniente eliminar el salto de línea.

Para ello utilizamos el método `strip()`.

```
with open("alumnos.txt", "r") as archivo:
    for linea in archivo:
        print(linea.strip())
```

Salida:

```
Juan
María
Pedro
Ana
```

`strip()` elimina los espacios y saltos de línea ubicados al comienzo y al final de la cadena.

9. Recorrer un archivo con un bucle

La forma más utilizada para leer un archivo es recorrerlo mediante un bucle `for`.

```
with open("alumnos.txt", "r") as archivo:
    for linea in archivo:
        print(linea.strip())
```

Python lee automáticamente una línea en cada iteración.

Esta es la forma más eficiente de procesar archivos grandes.

10. Convertir datos leídos

Los datos obtenidos desde un archivo siempre se leen como texto.

Si el archivo contiene números:

```
8
6
9
10
```

el programa obtiene:

```
"8"  
"6"  
"9"  
"10"
```

Si queremos realizar operaciones matemáticas, debemos convertirlos.

```
with open("notas.txt", "r") as archivo:  
    for linea in archivo:  
        nota = int(linea)  
        print(nota + 1)
```

Salida:

```
9  
7  
10  
11
```

11. Leer un archivo y almacenarlo en una lista

Es frecuente guardar los datos leídos en una lista para procesarlos posteriormente.

```
notas = []  
  
with open("notas.txt", "r") as archivo:  
    for linea in archivo:  
        notas.append(int(linea))  
  
print(notas)
```

Salida:

```
[8, 6, 9, 10]
```

Ahora la información puede utilizarse como cualquier otra lista de Python.

12. Comparación de los métodos de lectura

Método	¿Qué devuelve?	¿Cuándo utilizarlo?
<code>read()</code>	Todo el archivo como un <code>str</code>	Archivos pequeños
<code>readline()</code>	Una línea	Leer línea por línea manualmente
<code>readlines()</code>	Lista de líneas	Cuando se desea almacenar todas las líneas
<code>for linea in archivo</code>	Una línea por iteración	Archivos grandes o procesamiento secuencial

13. Ejercicios

Ejercicio 1

Crear un archivo llamado `numeros.txt` con cinco números enteros.

Leer el archivo utilizando `read()` y mostrar su contenido.

Ejercicio 2

Leer el mismo archivo utilizando `readline()` y mostrar las dos primeras líneas.

Ejercicio 3

Leer el archivo utilizando un bucle `for` y calcular la suma de todos los números almacenados.

Ejercicio 4

Crear un programa que lea un archivo con nombres de estudiantes y los almacene en una lista.

Luego mostrar la cantidad total de estudiantes registrados.

Ejercicio 5

Una escuela almacena las calificaciones finales de sus estudiantes en un archivo llamado `calificaciones.txt`, con una nota por línea.

Desarrollar un programa modular que:

- lea todas las calificaciones;
 - las almacene en una lista;
 - calcule el promedio;
 - determine la nota máxima y la mínima;
 - muestre un informe final con los resultados.
-

B2.5.3 – Escritura de archivos

1. ¿Qué significa escribir un archivo?

Escribir un archivo consiste en almacenar información generada por el programa dentro de un archivo del sistema.

Por ejemplo, un programa puede guardar:

- las notas de los estudiantes;
- los resultados de un examen;
- una lista de clientes;
- un historial de operaciones;
- un registro de errores.

De esta manera, la información permanecerá disponible incluso después de finalizar la ejecución del programa.

2. Abrir un archivo para escritura

Para escribir información se utiliza el modo:

```
"w"
```

Ejemplo:

```
with open("mensaje.txt", "w") as archivo:  
    archivo.write("Hola Mundo")
```

Después de ejecutar este programa se crea el archivo:

```
mensaje.txt
```

```
Hola Mundo
```

3. ¿Qué ocurre si el archivo ya existe?

El modo "w" **sobrescribe completamente** el contenido existente.

Supongamos que el archivo contiene:

```
Juan  
María  
Pedro
```

Si ejecutamos:

```
with open("alumnos.txt", "w") as archivo:  
    archivo.write("Ana")
```

El archivo quedará así:

```
Ana
```

Todo el contenido anterior se pierde.

4. El método `write()`

La escritura se realiza mediante:

```
archivo.write(texto)
```

Por ejemplo:

```
with open("saludo.txt", "w") as archivo:  
    archivo.write("Bienvenidos al curso de Python")
```

5. Escribir varias líneas

El método `write()` no agrega automáticamente un salto de línea.

Por ejemplo:

```
with open("nombres.txt", "w") as archivo:
    archivo.write("Juan")
    archivo.write("María")
```

Resultado:

JuanMaría

6. Agregar saltos de línea

Para escribir cada dato en una línea diferente debemos utilizar el carácter:

`\n`

Ejemplo:

```
with open("nombres.txt", "w") as archivo:
    archivo.write("Juan\n")
    archivo.write("María\n")
    archivo.write("Pedro\n")
```

Resultado:

Juan
María
Pedro

7. Escribir datos almacenados en una lista

Es muy frecuente guardar el contenido de una lista en un archivo.

```
notas = [8, 6, 9, 10]
with open("notas.txt", "w") as archivo:
    for nota in notas:
        archivo.write(str(nota) + "\n")
```

Resultado:

```
8
6
9
10
```

8. ¿Por qué utilizar `str()`?

El método `write()` únicamente puede escribir cadenas de texto.

Este código produce un error:

```
archivo.write(8)
```

Debe convertirse el número a texto:

```
archivo.write(str(8))
```

o

```
archivo.write(str(nota))
```

9. Escribir información ingresada por el usuario

```
nombre = input("Ingrese su nombre: ")
with open("usuario.txt", "w") as archivo:
    archivo.write(nombre)
```

Si el usuario escribe:

Lucía

El archivo contendrá:

Lucía

10. Escribir varias líneas utilizando un bucle

```
with open("numeros.txt", "w") as archivo:
    for i in range(1, 6):
        archivo.write(str(i) + "\n")
```

Resultado:

1
2
3
4
5

11. Flujo completo

Crear datos

↓

Abrir archivo ("w")

↓

Escribir información

↓

Cerrar automáticamente (with)

↓

Archivo almacenado

12. Comparación entre lectura y escritura

Lectura	Escritura
"r"	"w"
Recupera información	Guarda información
El archivo debe existir	Si no existe, se crea
No modifica el archivo	Sobrescribe el contenido existente

13. Ejercicios

Ejercicio 1

Crear un archivo llamado `saludo.txt` y escribir un mensaje de bienvenida.

Ejercicio 2

Solicitar al usuario cinco nombres y guardarlos en un archivo, uno por línea.

Ejercicio 3

Generar un archivo que contenga los números del 1 al 100, cada uno en una línea diferente.

Ejercicio 4

Crear una lista con las notas de varios estudiantes y almacenarla en un archivo de texto.

Ejercicio 5

Una escuela desea guardar las calificaciones obtenidas por un estudiante.

Desarrollar un programa modular que:

- solicite el nombre del estudiante;
- solicite cinco calificaciones;
- cree un archivo con el nombre del estudiante;
- almacene las cinco calificaciones, una por línea;

- muestre un mensaje indicando que la información fue guardada correctamente.
-

B2.5.4 – Agregar información a un archivo

1. ¿Qué significa agregar información?

En muchas aplicaciones no queremos reemplazar el contenido de un archivo, sino **añadir nuevos datos al final**.

Por ejemplo:

- Registrar nuevos estudiantes.
- Guardar las ventas realizadas durante el día.
- Registrar las acciones de un usuario (logs).
- Agregar nuevos resultados de un experimento.

En estos casos utilizamos el modo de apertura:

```
"a"
```

La letra **a** proviene de la palabra inglesa **append**, que significa **agregar**.

2. Diferencia entre "w" y "a"

Supongamos que el archivo **alumnos.txt** contiene:

```
Juan  
María  
Pedro
```

Si utilizamos:

```
with open("alumnos.txt", "w") as archivo:  
    archivo.write("Ana")
```

El resultado será:

```
Ana
```

Todo el contenido anterior desaparece.

Si utilizamos:

```
with open("alumnos.txt", "a") as archivo:  
    archivo.write("Ana")
```

Obtendremos:

```
Juan  
María  
Pedro  
Ana
```

El contenido anterior permanece intacto.

3. Abrir un archivo en modo append

La sintaxis es muy similar a la utilizada anteriormente.

```
with open("archivo.txt", "a") as archivo:  
    # agregar información
```

Si el archivo no existe, Python lo crea automáticamente.

4. Agregar una línea

```
with open("notas.txt", "a") as archivo:  
    archivo.write("10\n")
```

Si el archivo contenía:

```
8  
7  
9
```

Después de ejecutar el programa contendrá:

```
8  
7
```

```
9  
10
```

5. Agregar varias líneas

```
with open("nombres.txt", "a") as archivo:  
    archivo.write("Lucía\n")  
    archivo.write("Carlos\n")  
    archivo.write("Sofía\n")
```

Cada llamada a `write()` agrega información al final del archivo.

6. Agregar datos ingresados por el usuario

```
nombre = input("Ingrese un nombre: ")  
with open("alumnos.txt", "a") as archivo:  
    archivo.write(nombre + "\n")
```

Cada vez que el programa se ejecuta, el nuevo nombre queda registrado sin eliminar los anteriores.

7. Agregar elementos de una lista

```
productos = ["Mouse", "Teclado", "Monitor"]  
with open("productos.txt", "a") as archivo:  
    for producto in productos:  
        archivo.write(producto + "\n")
```

Resultado:

```
Mouse  
Teclado  
Monitor
```

Si el archivo ya contenía información, estos elementos se agregan al final.

8. Ejemplo práctico

Supongamos un sistema que registra las asistencias de los estudiantes.

Cada vez que un alumno llega al aula:

```
nombre = input("Nombre del estudiante: ")
with open("asistencias.txt", "a") as archivo:
    archivo.write(nombre + "\n")
print("Asistencia registrada.")
```

Después de varios ingresos, el archivo podría contener:

```
Juan
María
Pedro
Ana
Carlos
```

Cada ejecución del programa incorpora nuevos registros.

9. ¿Cuándo utilizar "a"?

El modo "a" es recomendable cuando:

- se registran nuevos datos constantemente;
 - no se desea perder la información existente;
 - se crean historiales o registros;
 - el archivo funciona como una base de datos sencilla.
-

10. Comparación de los modos de apertura

Modo	Si el archivo existe	Si el archivo no existe	Uso principal
"r"	Lo abre para leer	Produce un error	Lectura
"w"	Borra el contenido y escribe desde cero	Lo crea	Escritura
"a"	Conserva el contenido y agrega datos al final	Lo crea	Agregar información

11. Ejercicios

Ejercicio 1

Crear un archivo llamado `diario.txt`.

Agregar tres líneas utilizando el modo `"a"`.

Ejecutar el programa varias veces y observar cómo el contenido del archivo va creciendo.

Ejercicio 2

Solicitar al usuario cinco nombres y agregarlos a un archivo existente.

Ejercicio 3

Crear un programa que registre las temperaturas diarias de una semana en un archivo sin borrar las mediciones anteriores.

Ejercicio 4

Explicar en qué situaciones utilizarías:

- `"r"`
 - `"w"`
 - `"a"`
-

Ejercicio 5

Una biblioteca necesita registrar cada préstamo realizado.

Desarrollar un programa modular que:

- solicite el nombre del lector;
 - solicite el título del libro prestado;
 - registre ambos datos en un archivo llamado `prestamos.txt`;
 - agregue cada préstamo en una nueva línea sin eliminar los registros anteriores;
 - muestre un mensaje indicando que el préstamo fue registrado correctamente.
-

B2.5.5 – Manejo de errores en archivos

1. ¿Qué es un error en un programa?

Durante la ejecución de un programa pueden ocurrir situaciones inesperadas.

Por ejemplo:

- intentar abrir un archivo que no existe;
- intentar dividir un número por cero;
- ingresar texto cuando se esperaba un número;
- escribir en un archivo sin permisos.

Estas situaciones generan una **excepción** (*exception*).

Si no se controla, el programa finaliza mostrando un mensaje de error.

2. Ejemplo sin manejo de errores

Supongamos que intentamos abrir un archivo que no existe.

```
with open("estudiantes.txt", "r") as archivo:
    contenido = archivo.read()
print(contenido)
```

Si el archivo no existe, Python muestra un mensaje similar a:

```
FileNotFoundError:
[Errno 2] No such file or directory:
'estudiantes.txt'
```

El programa termina inmediatamente.

3. ¿Qué es una excepción?

Una excepción es un evento que interrumpe el flujo normal de ejecución de un programa.

Python permite detectar estas situaciones y decidir cómo actuar para evitar que el programa se cierre inesperadamente.

4. La estructura `try` y `except`

La forma más común de manejar errores es mediante:

```
try:
    # código que puede producir un error
except:
    # código que se ejecuta si ocurre un error
```

5. Primer ejemplo

```
try:
    with open("estudiantes.txt", "r") as archivo:
        print(archivo.read())
except:
    print("No fue posible abrir el archivo.")
```

Si el archivo existe:

```
Juan
María
Pedro
```

Si el archivo no existe:

```
No fue posible abrir el archivo.
```

El programa continúa ejecutándose normalmente.

6. Capturar errores específicos

Es recomendable indicar qué tipo de excepción esperamos.

```
try:
    with open("estudiantes.txt", "r") as archivo:
        print(archivo.read())
except FileNotFoundError:
    print("El archivo no existe.")
```

Esto hace que el programa sea más claro y evita ocultar otros errores.

7. Manejar varios tipos de excepciones

Un mismo bloque puede controlar distintos errores.

```
try:
    numero = int(input("Ingrese un número: "))
    resultado = 100 / numero
    print(resultado)
except ValueError:
    print("Debe ingresar un número entero.")
```

```
except ZeroDivisionError:
    print("No es posible dividir por cero.")
```

8. La cláusula `else`

El bloque `else` se ejecuta únicamente si no ocurre ninguna excepción.

```
try:
    archivo = open("datos.txt", "r")
except FileNotFoundError:
    print("Archivo inexistente.")
else:
    print("Archivo abierto correctamente.")
    archivo.close()
```

9. La cláusula `finally`

El bloque `finally` se ejecuta siempre, independientemente de que ocurra o no un error.

```
try:
    archivo = open("datos.txt", "r")
except FileNotFoundError:
    print("Archivo inexistente.")
finally:
    print("Fin del programa.")
```

Este bloque suele utilizarse para liberar recursos.

10. Esquema general

```
try
|
|— Código que puede producir un error
|
|— except
|   Manejo del error
|
|— else
|   Se ejecuta si no hubo errores
|
|— finally
|   Siempre se ejecuta
```

11. Ejemplo completo

```
try:
    nombre = input("Nombre del archivo: ")
    with open(nombre, "r") as archivo:
        print(archivo.read())
except FileNotFoundError:
    print("El archivo indicado no existe.")
else:
    print("Lectura realizada correctamente.")
finally:
    print("Programa finalizado.")
```

12. Errores comunes relacionados con archivos

Excepción	Situación
<code>FileNotFoundError</code>	El archivo no existe
<code>PermissionError</code>	No se tienen permisos para acceder al archivo

Excepción	Situación
<code>IsADirectoryError</code>	Se intentó abrir una carpeta como si fuera un archivo

13. Ejercicios

Ejercicio 1

Crear un programa que intente abrir un archivo inexistente utilizando `try` y `except`.

Mostrar un mensaje apropiado cuando el archivo no exista.

Ejercicio 2

Solicitar un número al usuario y manejar los siguientes errores:

- ingreso de texto;
 - división por cero.
-

Ejercicio 3

Modificar un programa de lectura de archivos para que informe un mensaje amigable cuando el archivo solicitado no exista.

Ejercicio 4

Explicar cuál es la función de cada uno de los siguientes bloques:

- `try`
 - `except`
 - `else`
 - `finally`
-

Ejercicio 5

Una escuela almacena las calificaciones de los estudiantes en archivos de texto.

Desarrollar un programa modular que:

- solicite el nombre del archivo;
- intente leer su contenido;
- informe un mensaje si el archivo no existe;

- muestre las calificaciones si la lectura fue exitosa;
 - indique al finalizar que el proceso ha concluido correctamente, independientemente de si ocurrió un error.
-